BEYOND CLICKOPZ

From Manual Cloud Deployments
to Infrastructure as Code



CHRISTOS
PANAGIOTIDIS

Beyond ClickOpz

From Manual Cloud Deployments on Azure to Infrastructure as Code

PUBLISHED By Christos Panagiotidis

© Copyright Christos Panagiotidis 2026 -All rights reserved.

The content contained within this book may not be reproduced, duplicated or transmitted without direct written permission from the author or the publisher.

Under no circumstances will any blame or legal responsibility be held against the publisher, or author, for any damages, reparation, or monetary loss due to the information contained within this book. Either directly or indirectly. You are responsible for your own choices, actions, and results.

Legal Notice:

This book is copyright protected. This book is only for personal use. You cannot amend, distribute, sell, use, quote or paraphrase any part, or the content within this book, without the consent of the author or publisher.

Disclaimer Notice:

Please note the information contained within this document is for educational and entertainment purposes only. All effort has been executed to present accurate, up to date, and reliable, complete information. No warranties of any kind are declared or implied. Readers acknowledge that the author is not engaging in the rendering of legal, financial, medical or professional advice. The content within this book has been derived from various sources. Please consult a licensed professional before attempting any techniques outlined in this book.

By reading this document, the reader agrees that under no circumstances is the author responsible for any losses, direct or indirect, which are incurred as a result of the use of the information contained within this document, including, but not limited to, — errors, omissions, or inaccuracies.

About the Author

Christos "Chris" Panagiotidis | Azure Cloud Engineer, MCT, Cloud Coach, and Lab Builder

Christos "Chris" Panagiotidis is a Germany-based Azure Cloud Engineer, Microsoft Certified Trainer, cloud coach, mentor, and hands-on lab builder. His public work sits at the intersection of Azure infrastructure, networking, automation, and practical education. He writes and builds for people who want to move beyond clicking through a portal and toward repeatable cloud systems that can be read, reviewed, tested, improved, and rebuilt with confidence.

Chris approaches cloud engineering as a craft that becomes stronger when it is practiced in the open. His GitHub profile describes his current role as Azure Cloud Engineer and MCT, and his focus areas include Azure infrastructure and networking, Terraform and Infrastructure as Code, cloud coaching and mentoring, and hands-on lab development. Those focus areas explain the structure of this book. The goal is not only to show that an Azure resource can be deployed. The goal is to teach the habit of turning a cloud idea into files, commands, review steps, validation checks, cleanup routines, and documentation that a learner or operator can trust.

His learning philosophy is direct: the best way to learn is by doing. That belief shapes his labs, his writing, and the way this book is organized. A reader should be able to see the architecture, inspect the Terraform, run the commands, validate the result, and understand why each piece exists. The important lesson is rarely a single resource. It is the relationship between naming, network boundaries, provider authentication, variables, state, outputs, monitoring, source control, and cleanup. Chris teaches those relationships by keeping examples concrete and by making the operational path visible.

On GitHub, Chris maintains and shares Azure lab repositories that support this same learning style. His public projects include an AKS landing zone lab, an Azure landing zone lab, an Azure networking lab for AZ-700 preparation, Terraform GitHub Actions workflows, and Azure From Zero To Hero. These repositories are not just code dumps. They are learning surfaces: README files, Terraform modules, workflow examples, validation notes, and operational patterns that help learners connect cloud theory with repeatable practice. The repositories also make the author's bias clear: infrastructure knowledge becomes more useful when it can be cloned, inspected, versioned, and improved.

Beyond Cloud with Chriz extends that work outside GitHub. Through the site, Chris publishes cloud training material, Azure news, certification paths, practical lab references, and short operational writeups. The site reflects the same purpose as the repositories: make cloud knowledge easier to act on. Cloud teams do not only need announcements or screenshots. They need to know what changed, what to test, what to document, what to automate, and what risk a

new service or feature introduces into a real environment. Chris writes for that practical layer between product news and day-to-day operations.

This book is a natural continuation of that work. ClickOps is often the place where cloud learning begins. The Azure portal is visual, useful, and important for discovery. But manual deployment alone does not preserve intent, review history, naming discipline, policy boundaries, or teardown safety. Chris wrote this manuscript to help readers cross that gap. The reader starts with familiar Azure building blocks and then learns how to express them as Terraform, how to validate them through command-line workflows, and how to think about the environment as something that can be rebuilt instead of merely remembered.

The tone of Chris's work is practical because the subject is practical. Azure skills are not proved by memorizing service names. They are proved by creating a working resource group, choosing the right network shape, keeping secrets out of source code, validating a plan before applying it, using outputs for useful inspection, and destroying lab resources when they are no longer needed. The same discipline appears in larger environments: hub-spoke networking, private endpoints, load balancing, scale sets, monitoring, CI pipelines, and secure operational handoff. Each chapter in this book is part of that progression.

Chris also writes as a coach. Coaching changes how technical material is explained. It is not enough for the author to know the answer. The reader needs a path that makes the answer discoverable. That means keeping examples small enough to reason about, naming variables clearly, repeating the command loop until it becomes habit, and showing the reason behind checks instead of hiding them as ceremony. It also means respecting beginners without lowering the bar. A learner can start with a simple lab and still practice professional habits from the first file.

The author's public profile also shows the toolchain behind his work: Azure, Terraform, PowerShell, Azure CLI, Git, GitHub, GitHub Actions, VS Code, Azure Monitor, Log Analytics, and related operations tooling. Those tools appear throughout the book because they are the practical bridge between cloud design and cloud operation. Infrastructure as Code is not only about writing resource blocks. It is about using a workflow that helps a human make better decisions before a cloud platform makes irreversible changes.

Most of all, Chris writes for people who are building confidence. A cloud learner can feel overwhelmed by service names, diagrams, certification paths, and vendor documentation that assumes too much too quickly. His work breaks that pressure into smaller moves: create the file, name the resource, run the check, read the plan, apply intentionally, inspect the result, and clean up. That rhythm is useful for beginners, but it is also useful for experienced operators

because it keeps the work honest. Every lab is a chance to practice the habits that protect real environments later.

Readers who want to follow Chris's current work can start with the links below. The bio page is the best public landing point for his training work and cloud writing. The GitHub profile is the best place to inspect the labs, repositories, and automation examples that sit behind this book. Together, they show the same message from two angles: cloud skills grow when learning is hands-on, visible, repeatable, and honest about the operational details.

Author Links

Bio: https://www.beyondcloudwithchriz.com/about

GitHub profile: https://github.com/jamonygr

LinkedIn: https://www.linkedin.com/in/christospanagiotidis/

Table of Contents

About the Author
From Clicks to Repeatable Cloud Builds
Chapter 0: Optional VS Code Terraform PowerShell Setup
Chapter 1: Foundations
Chapter 2: Windows Workstation Setup
Chapter 3: Terraform Core Workflow
Chapter 4: Provider Authentication
Chapter 5: Resource Groups And Tags
Chapter 6: Variables Locals Outputs
Chapter 7: State And Locking Basics
Chapter 8: Virtual Network Foundation
Chapter 9: Network Security Groups
Chapter 10: Windows VM Basics
Chapter 11: Windows VM IIS Bootstrap
Chapter 12: Azure Bastion RDP
Chapter 13: Standard Load Balancer Windows
Chapter 14: Load Balancer NAT Rules
Chapter 15: Count Windows VMs
Chapter 16: For Each Windows VMs
Chapter 17: Windows VMSS Manual Scaling
Chapter 18: Windows VMSS Autoscaling
Chapter 19: NAT Gateway Outbound
Chapter 20: Private DNS
Chapter 21: Public DNS Optional
Chapter 22: Remote State Storage
Chapter 23: Cross Environment State
Chapter 24: Traffic Manager Or Front Door
Chapter 25: Key Vault Secrets
Chapter 26: Private Endpoint Storage
Chapter 27: Azure Monitor Log Analytics
Chapter 28: GitHub Actions Terraform
Chapter 29: Azure SQL Private Access
Chapter 30: Final Windows Reference Architecture
Operating Azure as Code

From Clicks to Repeatable Cloud Builds

> **IMPORTANT REPOSITORY NOTICE: THIS BOOK IS BASED ON THE AZURE-FROM-ZERO-TO-HERO REPOSITORY: GITHUB.COM/JAMONYGR/AZURE-FROM-ZERO-TO-HERO. THE REPOSITORY IS THE SOURCE FOR THE LABS, TERRAFORM FILES, WINDOWS WORKSTATION SETUP, CHAPTER ORDER, AND REPEATABLE CLOUD BUILD WORKFLOW USED THROUGHOUT THIS BOOK.**

Manual Azure deployment is often the first useful doorway into the cloud. The portal is visual, immediate, and forgiving. It lets a person create a resource group, shape a virtual network, choose a Windows Server image, attach a public IP, and see the result without first learning an automation language. That matters. A beginner can understand the service before understanding the full build system.

This book is based on the GitHub repository [Jamonygr/Azure-From-Zero-To-Hero](https://github.com/Jamonygr/Azure-From-Zero-To-Hero). The repository is the technical foundation for the manuscript: its lab folders, Terraform files, diagrams, PowerShell runbooks, validation checks, and cleanup notes define the path the book follows. The goal of the book is to turn that working repository into a clear guided journey from manual Azure deployment to repeatable Infrastructure as Code.

The weakness of manual work appears when the same environment must be built again. A name changes slightly. A tag is missed. A subnet range is copied from memory instead of from a source file. A security rule is accepted because it seems reasonable in the moment. Afterward, no one can prove why that exact setting was chosen. The resource exists, but the decision trail is thin.

That is the hard part of click-built cloud work. The portal can show the current resource, but it does not naturally preserve a reviewed build path. It does not make repeatability automatic. It does not give a team a clean diff. It does not turn a lab into a versioned artifact that can be checked, shared, rerun, and safely removed.

The problem grows as the environment grows. This book does not stop with one resource group. It moves through Terraform workflow, provider authentication, naming, variables, state, virtual networks, Network Security Groups, Windows Server, IIS, Bastion, load balancing, VM scale sets, DNS, remote state, Key Vault, private endpoints, monitoring, GitHub Actions, Azure SQL, and a final Windows reference architecture. Manual deployment can teach those pieces, but it cannot manage that much detail with the same reliability as code.

Cleanup is another reason manual deployment breaks down. A lab can leave behind a VM, public IP, load balancer, storage account, database, monitoring workspace, or private endpoint. Those resources can continue to cost money and clutter the subscription. A serious lab path needs a reliable exit, not only an exciting build.

This book keeps the Azure portal in its proper place. It is useful for inspection, learning, and visibility. It is not the build system for this journey. The build system is Terraform, written in files, reviewed before apply, validated through commands, and cleaned up when the lesson is complete.

Infrastructure as Code changes the center of cloud work. Instead of treating the deployed resource as the only source of truth, it treats the configuration files as the source of truth. Terraform reads those files, compares them with the current Azure environment, and prepares a plan. That plan shows what Terraform intends to create, change, or destroy before the change is applied.

That review point is the first major shift. Manual deployment often jumps from intention to action. Terraform puts a plan between them. The plan can be read, questioned, saved, rejected, or applied. This does not remove the need for judgment. It gives judgment a better place to happen.

The second shift is consistency. Names can come from shared locals. Tags can be applied the same way in every lab. Inputs can live in variables. Safe examples can live in `terraform.tfvars.example`. Outputs can show values needed for validation. The same folder can be formatted, validated, planned, applied, inspected, and destroyed without relying on memory.

The operating loop is simple enough to remember and strict enough to matter:

```
terraform init
terraform fmt -check
terraform validate
terraform plan -out tfplan
terraform apply tfplan
terraform destroy
```

Each command has a job. `terraform init` prepares the working directory and provider dependencies. `terraform fmt -check` confirms that the files are formatted. `terraform validate` checks whether the configuration is structurally valid. `terraform plan -out tfplan` shows the proposed change and saves the reviewed plan. `terraform apply tfplan` applies that exact reviewed plan. `terraform destroy` removes the resources when the lab is done.

This loop appears throughout the book because it is the rhythm of safe Terraform work. It works for a small foundation lab and remains relevant when the path reaches VM scale sets, private endpoints, monitoring, CI, and a final architecture. The tools change the habit from "click and remember" to "write, check, plan, apply, and clean up."

The book follows the repository instead of inventing a separate learning path. Each `CLZ-*` lab folder becomes one chapter. Chapter 0 covers the optional workstation setup because Terraform depends on local tools before it can manage Azure resources. Chapter 1 starts the

core foundation path. Chapter 30 brings the work together in the final Windows reference architecture.

Every lab chapter follows the same practical movement: concept, architecture, code, validation, cleanup, and continuation. The repeated structure is intentional. It lets the reader focus on the changing Azure pattern instead of relearning the chapter format, while the chapter itself remains clean prose without visible internal planning labels.

The source files are part of the teaching. The README gives the goal, build summary, runbook, validation checklist, cleanup note, and continuation context. Terraform files hold the implementation. `variables.tf` defines inputs. `locals.tf` shapes naming and tag composition. `outputs.tf` exposes values used for validation. `terraform.tfvars.example` shows safe example values. Local override files and real secrets do not belong in the manuscript or in source control.

The diagrams are also part of the work. The curriculum map shows the route from setup to operations. Other diagrams show the Terraform workflow, workstation toolchain, resource naming, network foundation, Windows IIS VM, Bastion RDP, load-balanced Windows tier, remote state, private endpoint, monitoring, GitHub Actions, and capstone architecture. A diagram should help the reader connect the code to the Azure shape being built.

Code snippets in this book have a strict purpose. Terraform snippets must come from the repository unless they are clearly labeled as command examples. PowerShell snippets should match the Windows-first path. Secrets, generated passwords, subscription IDs, tenant IDs, and personal account details must stay out. The book is meant to be practical, but practical does not mean careless.

The checks live outside the chapter artifact. That keeps the book chapter clean for reading while preserving the review evidence elsewhere. Each chapter check confirms source alignment, snippet accuracy, no unsafe values, command accuracy, validation coverage, cleanup guidance, continuity, and duplicate control.

The first movement of the book is local readiness. PowerShell, Terraform, Azure CLI, Git, and VS Code form the working surface. If those tools are missing or misconfigured, every Azure lab becomes harder to trust. That is why the optional setup chapter is still treated as a full chapter in the book.

The second movement is Terraform foundation work. The early chapters introduce desired state, provider authentication, resource groups, tags, variables, locals, outputs, state, and locking. These ideas are not glamorous, but they carry the rest of the book. Naming, inputs, outputs, and state are the difference between a one-time demo and a repeatable lab library.

The third movement is Azure infrastructure. The book builds network foundations first, then adds security rules, Windows compute, IIS bootstrap, Bastion access, load balancing, NAT rules, `count`, `for_each`, VM scale sets, and autoscaling. The path starts with individual pieces and then shows patterns for scale and safer access.

The fourth movement adds platform and operational depth. NAT Gateway handles controlled outbound access. Private DNS gives internal name resolution. Public DNS remains optional. Remote state and cross-environment state move Terraform toward shared work. Traffic Manager or Front Door introduces global routing. Key Vault, private endpoints, Azure Monitor, GitHub Actions, and Azure SQL add security, delivery, and operations.

The final movement is composition. The capstone chapter brings the core patterns together through modules, VMSS, Bastion, Key Vault, monitoring, networking, and outputs. It is not presented as the only possible production architecture. It is a reference point that shows how small, readable labs can become a connected design.

That is the promise of the book: from clicks to files, from individual resources to connected systems, from uncertain cleanup to deliberate teardown, and from manual memory to source-controlled cloud change.

Chapter 0: Optional VS Code Terraform PowerShell Setup

> **IMPORTANT REPOSITORY NOTICE: THIS BOOK IS BASED ON THE AZURE-FROM-ZERO-TO-HERO REPOSITORY: GITHUB.COM/JAMONYGR/AZURE-FROM-ZERO-TO-HERO. THE REPOSITORY IS THE SOURCE FOR THE LABS, TERRAFORM FILES, WINDOWS WORKSTATION SETUP, CHAPTER ORDER, AND REPEATABLE CLOUD BUILD WORKFLOW USED THROUGHOUT THIS BOOK.**

The first step in a serious Infrastructure as Code path is not always a cloud resource. Sometimes it is the machine in front of the reader. A Terraform lab can be written correctly and still fail because the local terminal cannot find `terraform`, Azure CLI is signed in to the wrong subscription, VS Code opens the wrong shell, or Git is not ready for repository work. Those are not Azure architecture problems. They are workstation problems.

This chapter is based on the `CLZ-090-optional-vscode-terraform-powershell-setup` lab. The lab is optional because some readers already have a working environment. If `pwsh --version`, `terraform version`, `az version`, `git --version`, and `code --version` all work from PowerShell, the reader may be ready to move on. If any of them fail, stopping here is the faster path.

This chapter does not create Azure resources. That is deliberate. The goal is to prepare a Windows-focused, PowerShell-first workstation that can run the rest of the book without avoidable setup failures. The local machine must be able to open the repository, run Terraform, authenticate with Azure CLI, edit files in VS Code, and keep changes under Git control.

The core toolchain has five required tools. PowerShell 7 is the preferred terminal and automation shell. VS Code is the recommended editor for Terraform, Markdown, PowerShell, YAML, and GitHub Actions files. Terraform CLI is the engine that initializes, validates, plans, applies, and destroys infrastructure. Azure CLI handles sign-in and subscription inspection. Git tracks changes and connects the local work to GitHub.

WinGet is the preferred installation path when the workstation supports it. It gives the setup a repeatable command-line shape:

```
winget install --id Microsoft.PowerShell --source winget
winget install --id Git.Git --source winget
winget install --id Microsoft.VisualStudioCode --source winget
winget install --id Hashicorp.Terraform --source winget
winget install --exact --id Microsoft.AzureCLI
```

On a managed workstation, WinGet may be unavailable or blocked. That is not a reason to improvise around policy. It simply means the reader should use the official installers or the organization's approved software channel. The required result is not WinGet itself. The required result is a terminal where the expected tools are installed, visible, and usable.

After installation, PowerShell should be closed and reopened. This sounds minor, but it fixes a common source of confusion. Installers often update `PATH`, and an existing terminal session may not see the new command paths. A fresh terminal gives the validation checks a fair result.

The first validation is direct:

```
pwsh --version
git --version
code --version
terraform version
az version
```

Each command should print version information. These checks do not prove that every Azure deployment workflow will succeed, but they prove the local shell can find the tools. If one command is missing, the reader should fix that tool before touching Azure resources. A broken workstation makes correct infrastructure code look broken.

VS Code is recommended because the repository is both a codebase and a learning path. The reader will move between README files, Terraform files, PowerShell scripts, workflow files, and diagrams. Opening the repository root with `code .` gives VS Code the full context: file search, Git status, Markdown links, integrated terminal, workspace settings, and extension recommendations.

The recommended extensions match the files used throughout the book:

```
code --install-extension hashicorp.terraform
code --install-extension ms-vscode.powershell
code --install-extension ms-azuretools.vscode-azureresourcegroups
code --install-extension github.vscode-github-actions
code --install-extension yzhang.markdown-all-in-one
```

The repository already tracks this extension list in `.vscode/extensions.json`. It also includes workspace settings for Terraform formatting, PowerShell editing, Markdown wrapping, line endings, and trailing whitespace cleanup. The editor does not replace command-line validation, but it reduces friction while reading and editing the labs.

Azure CLI sign-in is the next workstation boundary. Terraform can use Azure CLI authentication for Azure deployments, so the active account and subscription must be clear before any real deployment. The reader should sign in and inspect the context:

```
az login
az account show
az account list --output table
```

If more than one subscription is available, the intended subscription should be selected explicitly:

```
az account set --subscription "<subscription-id-or-name>"
az account show
```

This check prevents a quiet but serious mistake. A Terraform configuration can be valid and still deploy to the wrong subscription if the Azure CLI context points somewhere unexpected. Subscription confirmation should become as normal as checking the current folder.

The `CLZ-090` folder includes a safe Terraform smoke test. Its `versions.tf` file requires Terraform `>= 1.6.0`. Its `providers.tf` file intentionally declares no cloud provider. Its variables describe the expected workstation profile: Windows, PowerShell, and Visual Studio Code. Its locals list required commands and recommended extensions. Its outputs expose that metadata and point toward the `CLZ-100-foundations` lab folder used after setup.

The smoke test is run from the optional setup folder:

```
terraform init
terraform fmt -check
terraform validate
terraform plan
```

The plan should not create Azure infrastructure. It may show output values because the folder exposes validation metadata. That is expected. The important result is that Terraform can initialize, check formatting, validate the configuration, and prepare a plan from the correct folder before the book starts deploying real resources.

The repository also provides a helper script:

```
..\scripts\Test-AzureFromZeroToHeroToolchain.ps1
```

From the repository root, the same script is run as:

```
.\scripts\Test-AzureFromZeroToHeroToolchain.ps1
```

The script checks required commands, prints version output where possible, checks recommended VS Code extensions when `code` is available, and warns if Azure CLI is not signed in. It does not install tools and does not create Azure resources. Its value is evidence. It confirms whether the workstation is ready or points to the exact local tool that still needs attention.

Folder discipline is the quiet habit that protects the rest of the book. Terraform uses the current folder as the root module. Running from the wrong folder can produce confusing plans, local state in the wrong place, or cleanup commands aimed at the wrong lesson. Before any serious Terraform command, the reader should use:

```
Get-Location
```

That small check keeps the terminal honest. It matters because the workflow soon moves from local validation into real Azure resource creation. The habit should be in place before the risk increases.

The readiness standard is simple. PowerShell 7 works. Terraform works. Azure CLI works. Git works. VS Code works. The repository opens from the command line. The recommended extensions are installed or visible as recommendations. Azure CLI shows the intended subscription. Terraform validates inside the optional setup folder. Once those checks pass, the workstation is no longer the main question.

That is the point of this optional setup chapter: remove local uncertainty so the rest of the book can concentrate on Infrastructure as Code.

Chapter 1: Foundations

This chapter is based on the `CLZ-100-foundations` lab. The lab creates one Azure resource group in the default `eastus2` region, applies the shared naming and tagging model, exposes validation outputs, and then expects cleanup through `terraform destroy`.

The manual version of this task is easy to underestimate. A person can open the Azure portal, create a resource group, choose a region, and add tags. The first run may look fine. The second run may use a slightly different name. The third may miss a tag. Another run may land in a different region because the setting was not reviewed. Small inconsistencies become the first signs of unmanaged cloud work.

Terraform turns that simple action into a repeatable pattern. The `.tf` files describe what should exist. The plan shows what Terraform intends to create. The apply step makes the reviewed plan real. The outputs give the reader something to check. The destroy command removes the lab when the lesson is complete. Nothing about this rhythm is limited to resource groups. The same rhythm scales to networks, Windows compute, Bastion, VM scale sets, private endpoints, monitoring, and complete reference architectures.

The lab folder has the standard Azure From Zero To Hero shape. `versions.tf` defines Terraform and provider requirements. `providers.tf` configures AzureRM. `variables.tf` declares shared inputs. `locals.tf` builds derived names and tags. `resource-group.tf` creates the resource group. `lab.tf` holds a lesson checkpoint. `outputs.tf` exposes validation values. `terraform.tfvars.example` provides safe input examples.

The provider block is short, but it carries an important boundary:

```
provider "azurerm" {
  features {}
  subscription_id = var.subscription_id
}
```

AzureRM requires the `features {}` block. The subscription is controlled through `var.subscription_id`. In the example values, `subscription_id` is `null`, which lets Azure CLI context handle the active subscription where supported. If a reader needs to pin a subscription explicitly, that value belongs in a local `terraform.tfvars` file. It should not be committed and should not be hard-coded into the shared lab.

The shared variables make the lab reusable. The defaults use `dev`, `eastus2`, `CLZ-100`, and `clz`. The same variables file also includes broader values used by other lab patterns, such as admin naming, CIDR examples, instance count, DNS placeholders, remote state placeholders, and SQL administrator naming. This resource group lab does not need all of them. The point is to introduce a stable interface that can survive beyond one resource type.

The naming model is built in `locals.tf`. With the default values, `local.prefix` becomes `clz-dev-clz100`. That prefix is not random decoration. It carries the project prefix, environment, and lab ID. When Azure resources appear in the portal, the name should tell the reader which lesson created them.

The tag model is just as important:

```
tags = {
  Project     = "Azure From Zero To Hero"
  Environment = var.environment
  ManagedBy   = "Terraform"
  Lab         = var.lab_id
}
```

Tags make the lesson easier to audit. They identify the project, environment, management method, and lab owner. In a small lab, that may feel obvious. In a larger subscription, tags help with cleanup, cost review, ownership, and troubleshooting. This is why the tagging habit starts before the book introduces more expensive resources.

The resource group itself is created in `resource-group.tf`:

```
resource "azurerm_resource_group" "lab" {
  name     = "${local.prefix}-rg"
  location = var.location
  tags     = local.tags
}
```

This single block shows the core data flow. The resource name is derived from `local.prefix`. The location comes from `var.location`. The tags come from `local.tags`. Terraform uses those references to understand how values move through the configuration. The reader is not only creating a resource group. The reader is learning how Terraform files connect.

The outputs keep validation concrete:

```
output "resource_group_name" {
  description = "Resource group created by this lab."
  value       = azurerm_resource_group.lab.name
}
output "location" {
  description = "Azure region used by this lab."
  value       = azurerm_resource_group.lab.location
}
```

After apply, these outputs should match the planned resource. The resource group name should begin with `clz-dev-clz100`, and the location should be `eastus2` unless the reader intentionally changed it with a local override. This is a small validation step, but it creates the habit the rest of the book depends on: do not simply run Terraform; confirm what it did.

The command loop stays disciplined:

```
terraform init
terraform fmt -check
terraform validate
terraform plan -out tfplan
terraform apply tfplan
terraform output
terraform destroy
```

The expected plan is narrow. It should show one resource group, standard tags, and the expected region. If Terraform plans additional resources, the reader is probably in the wrong folder or using unexpected local values. That is a reason to stop, not a reason to apply faster.

Cleanup belongs to the same folder. Running `terraform destroy` from `CLZ-100-foundations` removes the resource group managed by that lesson's state. This boundary matters because the repository is organized as independently deployable labs. Each lesson should be understandable, runnable, and removable on its own.

The chapter is small by design. A tagged resource group teaches the method without hiding it behind service complexity. The reader sees provider setup, variables, locals, a resource block, outputs, a plan, validation, and cleanup in one place. More Azure services can be added without changing the foundation: clear files, predictable names, standard tags, reviewed plans, and disciplined teardown.

Chapter 2: Windows Workstation Setup

This chapter is based on the `CLZ-110-windows-workstation-setup` lab. The lab focuses on Terraform, Azure CLI, VS Code, Git, and PowerShell as one working environment. It creates one tagged Azure resource group so the workstation can be checked with a real Azure deployment.

A workstation can look ready and still be wrong. Terraform may be installed, but PowerShell may be opened in the wrong folder. Azure CLI may be signed in, but the active subscription may be the wrong one. VS Code may open, but not at the repository root. Git may work, but local-only files may still need attention. These are not advanced cloud problems. They are local context problems, and they can make correct Terraform look broken.

The lab keeps the Azure scope small on purpose. It creates one resource group in `eastus2` with a name that starts with `clz-dev-clz110`. The resource group is a controlled test of the whole local path: provider initialization, Azure CLI authentication, subscription context, naming locals, tags, outputs, and cleanup from the same lesson folder.

The file shape stays familiar. `versions.tf` requires Terraform `>= 1.6.0` and declares AzureRM `~> 4.0` plus random `~> 3.6`. `providers.tf` configures AzureRM. `variables.tf` declares shared inputs. `locals.tf` builds names and tags. `resource-group.tf` creates the resource group. `lab.tf` records the lesson checkpoint. `outputs.tf` exposes validation values. `terraform.tfvars.example` provides safe examples for local use.

The provider block is where Terraform meets the local Azure context:

```
provider "azurerm" {
  features {}
  subscription_id = var.subscription_id
}
```

The `subscription_id` input is intentionally variable-driven. With the example value `null`, Terraform can use the active Azure CLI subscription where supported. That makes the local Azure CLI context part of the deployment path. If the wrong account or subscription is active, Terraform may behave correctly while still targeting the wrong place.

Before running a plan, the reader should inspect Azure CLI context:

```
az account show
az account list --output table
```

If the active subscription is not the intended one, it should be selected explicitly:

```
az account set --subscription "<subscription-id-or-name>"
```

This habit belongs early in the book. Subscription context is not a detail to check after something goes wrong. It is part of the readiness check before Terraform is allowed to make a change.

The lab's default values keep the run predictable. The safe example uses `environment = "dev"`, `location = "eastus2"`, `lab_id = "CLZ-110"`, `name_prefix = "clz"`, and `subscription_id =

null`. If the reader needs local overrides, `terraform.tfvars.example` can be copied to `terraform.tfvars`, but that local file must stay out of commits.

The naming and tag model comes from locals. With default values, the resource group name becomes `clz-dev-clz110-rg`. The tags identify the project, environment, management method, and lab:

```
tags = {
  Project     = "Azure From Zero To Hero"
  Environment = var.environment
  ManagedBy   = "Terraform"
  Lab         = var.lab_id
}
```

Those values are not cosmetic. They make the resource group recognizable in the Azure portal, support cleanup, and connect the deployed object back to the lesson that created it.

The Azure resource remains intentionally simple:

```
resource "azurerm_resource_group" "lab" {
  name     = "${local.prefix}-rg"
  location = var.location
  tags     = local.tags
}
```

This block proves that Terraform can take values from variables and locals, authenticate through the provider, and create a tagged Azure object. If this fails, the most likely issue is not a complex architecture mistake. It is usually provider setup, Azure CLI context, subscription access, network access, or folder discipline.

The outputs make validation explicit:

```
output "resource_group_name" {
  description = "Resource group created by this lab."
  value       = azurerm_resource_group.lab.name
}
output "location" {
  description = "Azure region used by this lab."
  value       = azurerm_resource_group.lab.location
}
```

After apply, `terraform output` should show the resource group name and region. The expected name begins with `clz-dev-clz110`, and the expected location is `eastus2` unless the reader intentionally changed the local values.

The command loop is the same disciplined loop, now tied to workstation confidence:

```
terraform init
terraform fmt -check
terraform validate
terraform plan -out tfplan
terraform apply tfplan
terraform output
terraform destroy
```

All commands should be run from `CLZ-110-windows-workstation-setup`. That folder boundary matters. Terraform treats the current folder as the root module. Running from the wrong folder can create the wrong state, plan the wrong lesson, or destroy the wrong lab.

The chapter is complete only after cleanup. Creating the resource group proves the workstation can reach Azure. Destroying it proves the reader can remove lab resources cleanly from the same folder. Both sides matter because this book treats cleanup as part of the workflow, not as an afterthought.

Once this lab is validated, the reader has more than installed tools. The reader has a Windows workstation that can run Terraform, use Azure CLI context, create an Azure resource, inspect outputs, and clean up safely. The workstation is ready for the core Terraform workflow: initialize, format, validate, plan, apply, inspect, and destroy.

Chapter 3: Terraform Core Workflow

This chapter is based on the `CLZ-120-terraform-core-workflow` lab. The lab teaches the core Terraform routine used throughout the book: initialize the folder, check formatting, validate the configuration, create a saved plan, apply the reviewed plan, inspect outputs, and destroy the lab resources.

Manual Azure changes do not naturally provide that routine. A resource can be created in the portal and still leave no dependable record of the exact choices behind it. A later change can happen without review. A cleanup task can depend on memory. The result may work, but the process is weak because it is difficult to repeat, inspect, and hand to another operator.

Terraform makes the process explicit. The files describe the intended state. The plan shows the proposed change before Azure is modified. The apply step turns a reviewed plan into infrastructure. The outputs give concrete validation values. The destroy command removes the lesson resources from the same working folder.

The lab keeps the Azure scope deliberately small: one tagged resource group in `eastus2`, named from the `clz-dev-clz120` prefix. That small scope keeps the workflow visible. The lesson is not the resource group itself. The lesson is the command discipline around it.

The folder uses the standard lab layout. `versions.tf` sets the Terraform and provider constraints. `providers.tf` configures AzureRM. `variables.tf` declares shared inputs. `locals.tf` builds names and tags. `resource-group.tf` creates the Azure resource group. `lab.tf` records the lesson checkpoint. `outputs.tf` exposes validation values. `terraform.tfvars.example` provides safe examples for local override values.

The workflow starts with provider preparation. `versions.tf` tells Terraform which core version and providers the folder expects:

```
terraform {
  required_version = ">= 1.6.0"
  required_providers {
    azurerm = {
      source  = "hashicorp/azurerm"
      version = "~> 4.0"
    }
    random = {
      source  = "hashicorp/random"
      version = "~> 3.6"
    }
  }
}
```

`terraform init` reads that block and prepares the working directory. If initialization fails, the rest of the workflow stops. A plan created from a half-prepared folder is not a reliable plan.

The AzureRM provider keeps the subscription choice variable-driven:

```
provider "azurerm" {
  features {}
  subscription_id = var.subscription_id
```

```
}
```

With the example value `subscription_id = null`, Terraform can use the active Azure CLI subscription where supported. That behavior is useful, but it also means local context matters. The workflow is not only about Terraform syntax. It is also about making sure Terraform is pointed at the intended Azure subscription before a plan is trusted.

The safe example values keep the lab predictable. `terraform.tfvars.example` uses `environment = "dev"`, `location = "eastus2"`, `lab_id = "CLZ-120"`, `name_prefix = "clz"`, and `subscription_id = null`. If a reader needs local overrides, the example file can be copied to `terraform.tfvars`, but real local values must stay out of commits.

The naming and tag model is built once in `locals.tf`:

```
locals {
  lab_number     = replace(var.lab_id, "CLZ-", "")
  lab_code       = lower(replace(var.lab_id, "CLZ-", "clz"))
  prefix         = lower("${var.name_prefix}-${var.environment}-${local.lab_code}")
  compact_prefix = substr(replace(local.prefix, "-", ""), 0, 18)
  tags = {
    Project     = "Azure From Zero To Hero"
    Environment = var.environment
    ManagedBy   = "Terraform"
    Lab         = var.lab_id
  }
}
```

With default values, `local.prefix` becomes `clz-dev-clz120`. The tags connect the Azure object back to the project, environment, management method, and lab. That makes the plan easier to review and the deployed resource easier to recognize.

The Azure resource is intentionally plain:

```
resource "azurerm_resource_group" "lab" {
  name     = "${local.prefix}-rg"
  location = var.location
  tags     = local.tags
}
```

This block gives Terraform one real Azure object to manage. The expected plan should show `azurerm_resource_group.lab` with the name `clz-dev-clz120-rg`, the location `eastus2`, and the standard tags. A plan that shows unrelated resources, a different name, missing tags, or an unexpected region should not be applied.

The lesson checkpoint in `lab.tf` keeps the purpose clear:

```
locals {
  lesson_checkpoint = "Run the Terraform workflow against this resource group."
}
```

That local value does not create cloud infrastructure. It documents the lab intention inside the Terraform folder. The command loop is the focus, and the resource group is the object used to practice it.

The outputs make validation concrete:

```
output "resource_group_name" {
  description = "Resource group created by this lab."
  value       = azurerm_resource_group.lab.name
```

```
}
output "location" {
  description = "Azure region used by this lab."
  value       = azurerm_resource_group.lab.location
}
```

After apply, `terraform output` should confirm the resource group name and region. Those outputs are small, but they teach an important habit: validation should come from the deployed state, not from memory of what was supposed to happen.

The runbook is the core of the chapter:

```
terraform init
terraform fmt -check
terraform validate
terraform plan -out tfplan
terraform apply tfplan
terraform output
terraform destroy
```

All commands should be run from `CLZ-120-terraform-core-workflow`. Terraform treats the current folder as the root module. Running from the wrong folder can initialize the wrong configuration, plan the wrong lab, or destroy resources that do not belong to the current lesson.

Each command is a gate. `terraform init` prepares providers. `terraform fmt -check` confirms consistent file formatting. `terraform validate` checks whether the configuration is valid. `terraform plan -out tfplan` calculates and saves the proposed change. `terraform apply tfplan` applies the reviewed plan. `terraform output` prints values used for validation. `terraform destroy` removes the lab resources.

The saved plan is especially important. It separates review from deployment. The reader should inspect the plan before applying it and confirm that it matches the README checklist: resource names start with `clz-dev-clz120`, tags include `Project`, `Environment`, `ManagedBy`, and `Lab`, outputs match the resource being created, and no local secrets are committed.

The workflow also defines when to stop. A formatting failure should be fixed before validation. A validation failure should be fixed before planning. A plan with unexpected changes should be investigated before apply. A completed apply should be followed by output checks and cleanup. Skipping those gates turns Terraform back into a faster version of manual clicking.

Cleanup is part of the workflow, not a separate chore. The README names `terraform destroy` as the cleanup path, and the command should run from the same lab folder. The destroy plan should point to the Chapter 3 resource group. After cleanup, generated local artifacts such as real `terraform.tfvars`, local state, and saved plan files should not be committed.

By the end of the chapter, the reader has a working command model for the rest of the book. Networking, Windows compute, private access, monitoring, CI, and reference architectures all become easier to manage when the operating loop stays recognizable: initialize, format,

validate, plan, review, apply, inspect, and destroy. The resource may change, but the review habit stays the same.

Chapter 4: Provider Authentication

This chapter is based on the `CLZ-130-provider-authentication` lab. The lab shows how Terraform uses the AzureRM provider, Azure CLI authentication, and subscription context before it can safely manage Azure resources.

Authentication problems often look like Terraform problems at first. A configuration can be valid, the command syntax can be correct, and the working folder can be right, but the active Azure identity can still point at the wrong subscription. That makes provider authentication a safety boundary. Terraform must not only run; it must run against the intended Azure subscription.

The lab keeps the Azure footprint small so that boundary is easy to inspect. It creates one tagged resource group in `eastus2` with a name that starts with `clz-dev-clz130`. The resource group proves that the AzureRM provider can authenticate, select the intended subscription context, create a controlled Azure object, expose outputs, and clean up from the same folder.

The folder uses the standard lab layout. `versions.tf` declares Terraform and provider constraints. `providers.tf` configures AzureRM. `variables.tf` defines shared inputs, including `subscription_id`. `locals.tf` builds names and tags. `resource-group.tf` creates the lab resource group. `lab.tf` records the lesson checkpoint. `outputs.tf` exposes validation values. `terraform.tfvars.example` provides safe example inputs.

Provider authentication begins with provider installation. The requirements in `versions.tf` tell Terraform which provider packages belong to this folder:

```
terraform {
  required_version = ">= 1.6.0"
  required_providers {
    azurerm = {
      source  = "hashicorp/azurerm"
      version = "~> 4.0"
    }
    random = {
      source  = "hashicorp/random"
      version = "~> 3.6"
    }
  }
}
```

`terraform init` reads this block and prepares the provider plugins. Initialization does not prove that Azure authentication is correct, but it is the first gate. Terraform cannot validate provider-backed configuration or produce a useful Azure plan until the provider runtime is available.

The AzureRM provider configuration is short and deliberate:

```
provider "azurerm" {
  features {}
  subscription_id = var.subscription_id
}
```

The `features {}` block is required by the AzureRM provider. The `subscription_id` argument is the important decision point. It makes the subscription target part of the Terraform configuration instead of leaving it as an unspoken assumption.

The variable definition explains the two supported paths:

```
variable "subscription_id" {
  description = "Azure subscription ID. Leave null to use the active Azure CLI subscription
when supported."
  type        = string
  default     = null
}
```

An explicit value can point Terraform at a known subscription. The default `null` value can let Terraform use the active Azure CLI subscription where supported. Both approaches are valid in the right context. The unsafe pattern is not knowing which path is being used.

Before a plan is trusted, the Azure CLI context should be inspected:

```
az account show
az account list --output table
```

If the active subscription is not the intended target, it should be changed before Terraform plans or applies:

```
az account set --subscription "<subscription-id-or-name>"
```

These are command examples, not values to commit. Azure CLI output can contain account-specific information, tenant details, and subscription identifiers. The reader should use that output locally to confirm context, then keep those values out of the manuscript and repository.

The safe example file keeps the lab reproducible without exposing a real account:

```
environment     = "dev"
location        = "eastus2"
lab_id          = "CLZ-130"
name_prefix     = "clz"
subscription_id = null
```

If a real subscription ID is needed for a local run, it belongs in a local `terraform.tfvars` file that is not committed. The example file documents the expected shape; it is not a place for private tenant, subscription, or credential values.

The naming and tag locals make the plan easy to recognize:

```
locals {
  lab_number     = replace(var.lab_id, "CLZ-", "")
  lab_code       = lower(replace(var.lab_id, "CLZ-", "clz"))
  prefix         = lower("${var.name_prefix}-${var.environment}-${local.lab_code}")
  compact_prefix = substr(replace(local.prefix, "-", ""), 0, 18)
  tags = {
    Project     = "Azure From Zero To Hero"
    Environment = var.environment
    ManagedBy   = "Terraform"
    Lab         = var.lab_id
  }
}
```

With the default values, `local.prefix` becomes `clz-dev-clz130`. The tags identify the project, environment, management method, and lab. During plan review, those values help confirm that Terraform is using the correct lesson inputs.

The Azure object stays intentionally simple:

```
resource "azurerm_resource_group" "lab" {
  name     = "${local.prefix}-rg"
  location = var.location
  tags     = local.tags
}
```

The expected plan should show one `azurerm_resource_group.lab` resource named `clz-dev-clz130-rg` in `eastus2` with the standard tags. If the plan shows a different name, unexpected resources, a different region, or missing tags, the workflow should stop before apply. If the Azure CLI context is wrong, the configuration may still be valid while the target subscription is wrong.

The lesson checkpoint remains plain:

```
locals {
  lesson_checkpoint = "Run the Terraform workflow against this resource group."
}
```

That local value documents the purpose of the folder. The lab is not trying to hide authentication inside a large architecture. It is using one resource group to make the provider boundary visible.

The outputs provide the validation surface:

```
output "resource_group_name" {
  description = "Resource group created by this lab."
  value       = azurerm_resource_group.lab.name
}
output "location" {
  description = "Azure region used by this lab."
  value       = azurerm_resource_group.lab.location
}
```

After apply, `terraform output` should confirm the resource group name and location. Those outputs should match the saved plan that was reviewed. They should not be used as a substitute for checking identity before the plan.

The Terraform workflow remains familiar:

```
terraform init
terraform fmt -check
terraform validate
terraform plan -out tfplan
terraform apply tfplan
terraform output
terraform destroy
```

All commands should be run from `CLZ-130-provider-authentication`. `terraform validate` confirms that the configuration is valid, but it does not prove the active Azure subscription is the intended one. Plan review in this chapter includes both the resource change and the identity context behind that change.

The saved plan protects the boundary between review and deployment. The reviewed plan should include one resource group to add, no unrelated changes, the `clz-dev-clz130-rg` name, the `eastus2` location, and the expected tags. The apply should use the saved `tfplan`, not an unreviewed fresh calculation.

Cleanup follows the same authentication rule. `terraform destroy` should run from the Chapter 4 folder, and the destroy plan should refer only to the Chapter 4 lab resource. A destroy command is still an authenticated Azure operation, so it deserves the same subscription awareness as apply.

After cleanup, local artifacts should remain local. Real `terraform.tfvars` files, state files, saved plan files, Azure CLI account output, tenant IDs, subscription IDs, client secrets, and credentials should not be committed. The source repository should keep examples and reusable configuration, not personal access details.

By the end of the chapter, the reader has a clear model for where Terraform authentication comes from. Terraform configuration defines the provider. Azure CLI or explicit input supplies the subscription context. The plan reveals what Terraform intends to do with that context. Names, tags, governance, and cleanup are easier to trust when every planned change is tied to an explicit Azure identity and subscription.

Chapter 5: Resource Groups And Tags

This chapter is based on the `CLZ-140-resource-groups-tags` lab. The lab standardizes resource group naming and tags so Azure resources are easier to identify, govern, cost-review, and clean up.

Manual Azure naming usually becomes inconsistent one small decision at a time. One resource group may use a clear prefix. Another may use a shortened project name. A tag may be added during one deployment and missed during the next. None of those choices feels serious alone, but together they make the environment harder to search, audit, and remove safely.

Terraform gives naming and tagging a stable place in the configuration. Instead of typing labels directly into every resource, the lab builds names and tags from variables and locals. That makes the pattern visible, reviewable, and repeatable. The naming model becomes part of the infrastructure design rather than a memory exercise.

The lab creates one tagged resource group in `eastus2` with a name that starts with `clz-dev-clz140`. The Azure footprint is intentionally small. The important lesson is the labeling model: resources should show the project, environment, management method, and lab context without requiring someone to guess.

The folder uses the standard lab layout. `versions.tf` declares Terraform and provider constraints. `providers.tf` configures AzureRM. `variables.tf` defines inputs such as environment, location, lab ID, and name prefix. `locals.tf` composes names and tags. `resource-group.tf` creates the resource group. `lab.tf` records the lesson checkpoint. `outputs.tf` exposes validation values. `terraform.tfvars.example` provides safe example inputs.

The naming model starts with input variables:

```
variable "environment" {
  description = "Deployment environment name."
  type        = string
  default     = "dev"
  validation {
    condition     = contains(["dev", "test", "prod"], var.environment)
    error_message = "Use dev, test, or prod."
  }
}
variable "location" {
  description = "Azure region for this lab."
  type        = string
  default     = "eastus2"
}
variable "lab_id" {
  description = "Azure From Zero To Hero lesson ID."
  type        = string
  default     = "CLZ-140"
}
variable "name_prefix" {
  description = "Short prefix used in Azure resource names."
  type        = string
  default     = "clz"
```

```
}
```

The environment validation is small but useful. It limits the value to `dev`, `test`, or `prod`, which prevents casual spelling differences from becoming permanent resource names and tags. Consistent inputs produce consistent Azure objects.

The safe example file shows the intended default values:

```
environment     = "dev"
location        = "eastus2"
lab_id          = "CLZ-140"
name_prefix     = "clz"
subscription_id = null
```

Those values produce the `clz-dev-clz140` prefix. If local overrides are needed, `terraform.tfvars.example` can be copied to `terraform.tfvars`, but real local values should stay out of commits. The example file documents the shape of the configuration without exposing private account details.

The main pattern lives in `locals.tf`:

```
locals {
  lab_number     = replace(var.lab_id, "CLZ-", "")
  lab_code       = lower(replace(var.lab_id, "CLZ-", "clz"))
  prefix         = lower("${var.name_prefix}-${var.environment}-${local.lab_code}")
  compact_prefix = substr(replace(local.prefix, "-", ""), 0, 18)
  tags = {
    Project     = "Azure From Zero To Hero"
    Environment = var.environment
    ManagedBy   = "Terraform"
    Lab         = var.lab_id
  }
}
```

This block turns a few input values into a reusable identity model. `local.lab_number` extracts `140` from `CLZ-140`. `local.lab_code` turns the lab ID into `clz140`. `local.prefix` combines the project prefix, environment, and lab code into `clz-dev-clz140`. `local.compact_prefix` removes hyphens and limits the value for Azure services that require shorter names.

The tag map is the governance signal. `Project` groups the resource with the book environment. `Environment` separates `dev`, `test`, and `prod` resources. `ManagedBy` makes Terraform ownership explicit. `Lab` ties the resource back to `CLZ-140`. These tags make search, cost review, and cleanup easier without requiring external notes.

The resource group consumes the composed values:

```
resource "azurerm_resource_group" "lab" {
  name     = "${local.prefix}-rg"
  location = var.location
  tags     = local.tags
}
```

With default inputs, Terraform plans `clz-dev-clz140-rg` in `eastus2`. The resource block stays clean because the naming and tag rules are not duplicated inside it. The resource uses a composed name and a shared tag map, which gives additional resources the same reusable identity model.

The outputs provide a simple validation surface:

```
output "resource_group_name" {
  description = "Resource group created by this lab."
  value       = azurerm_resource_group.lab.name
}
output "location" {
  description = "Azure region used by this lab."
  value       = azurerm_resource_group.lab.location
}
```

After apply, `terraform output` should confirm the resource group name and location. The output should match the reviewed plan. That gives the reader a quick check that the naming model reached Azure as expected.

The lab uses the standard Terraform command loop:

```
terraform init
terraform fmt -check
terraform validate
terraform plan -out tfplan
terraform apply tfplan
terraform output
terraform destroy
```

All commands should be run from `CLZ-140-resource-groups-tags`. For this chapter, plan review should focus on labels. The expected plan is one `azurerm_resource_group.lab`, the name `clz-dev-clz140-rg`, the location `eastus2`, and the standard tags `Project`, `Environment`, `ManagedBy`, and `Lab`.

A naming or tag mismatch should stop the workflow before apply. A wrong prefix may mean the wrong folder, wrong variable values, or a copied local override. A missing tag may mean the shared local map was changed or bypassed. These are small problems while the lab has one resource group, but they become expensive habits in larger designs.

Cleanup benefits from the same structure. A predictable name and `Lab = "CLZ-140"` make the resource easier to identify before destroy. `terraform destroy` should run from the same folder, and the destroy plan should refer only to the Chapter 5 resource group. After cleanup, generated local files such as real `terraform.tfvars`, state, and saved plan files should not be committed.

By the end of the chapter, naming and tagging are no longer cosmetic details. They are part of operating Azure as code. The same pattern also makes reusable Terraform configuration easier because variables, locals, and outputs can carry intent without duplicating values across files.

Chapter 6: Variables Locals Outputs

This chapter is based on the `CLZ-150-variables-locals-outputs` lab. The lab explains how Terraform separates inputs, derived values, reported values, and safe example overrides.

Manual Azure work often blends those concerns together. A region is chosen, a name is typed, a tag is added, and a useful result is remembered after deployment. That can work once, but it does not create a reusable model. Terraform becomes easier to maintain when each value has a clear role.

Variables accept input. Locals compute reusable values from inputs and rules. Outputs report values that matter after Terraform evaluates or deploys the configuration. `terraform.tfvars.example` documents safe example inputs without storing private local settings. That separation is the main lesson of the chapter.

The lab creates one tagged resource group in `eastus2` with a name that starts with `clz-dev-clz150`. The Azure object stays small so the value flow is easy to inspect. The same pattern supports networks, Windows compute, private access, monitoring, and complete architectures.

The folder uses the standard lab layout. `versions.tf` declares Terraform and provider constraints. `providers.tf` configures AzureRM. `variables.tf` defines input values. `locals.tf` computes names and tags. `resource-group.tf` creates the resource group. `lab.tf` records the lesson checkpoint. `outputs.tf` exposes validation values. `terraform.tfvars.example` provides safe example inputs.

The provider block shows a variable controlling provider behavior:

```
provider "azurerm" {
  features {}
  subscription_id = var.subscription_id
}
```

The matching input is defined in `variables.tf`:

```
variable "subscription_id" {
  description = "Azure subscription ID. Leave null to use the active Azure CLI subscription
when supported."
  type        = string
  default     = null
}
```

This variable keeps account-specific values out of the committed configuration. When it remains `null`, Terraform can use the active Azure CLI subscription where supported. When a local run needs an explicit subscription, that value belongs in a local override file that is not committed.

The environment input includes validation:

```
variable "environment" {
  description = "Deployment environment name."
  type        = string
  default     = "dev"
  validation {
    condition     = contains(["dev", "test", "prod"], var.environment)
```

```
    error_message = "Use dev, test, or prod."
  }
}
```

This validation protects the naming and tagging model. Unsupported values are rejected before they become resource names or tag values. That keeps `dev`, `test`, and `prod` consistent across labs.

The lab identity comes from a small set of direct inputs:

```
variable "location" {
  description = "Azure region for this lab."
  type        = string
  default     = "eastus2"
}
variable "lab_id" {
  description = "Azure From Zero To Hero lesson ID."
  type        = string
  default     = "CLZ-150"
}
variable "name_prefix" {
  description = "Short prefix used in Azure resource names."
  type        = string
  default     = "clz"
}
```

These values are intentionally simple. The point is not that every input is complex. The point is that values meant to vary are declared in one place, typed, described, and given defaults where defaults make sense.

The safe example file shows how local values are shaped:

```
environment     = "dev"
location        = "eastus2"
lab_id          = "CLZ-150"
name_prefix     = "clz"
admin_username  = "clzadmin"
admin_cidr      = "203.0.113.10/32"
instance_count  = 2
subscription_id = null
```

The example includes reusable lab values, not real secrets. If a reader copies it to `terraform.tfvars`, the copied file becomes local working input and should not be committed. The example file is the shared reference; the local override file is the machine-specific layer.

Locals turn inputs into reusable derived values:

```
locals {
  lab_number     = replace(var.lab_id, "CLZ-", "")
  lab_code       = lower(replace(var.lab_id, "CLZ-", "clz"))
  prefix         = lower("${var.name_prefix}-${var.environment}-${local.lab_code}")
  compact_prefix = substr(replace(local.prefix, "-", ""), 0, 18)
  tags = {
    Project     = "Azure From Zero To Hero"
    Environment = var.environment
    ManagedBy   = "Terraform"
    Lab         = var.lab_id
  }
}
```

With default inputs, `local.lab_number` becomes `150`, `local.lab_code` becomes `clz150`, and `local.prefix` becomes `clz-dev-clz150`. `local.compact_prefix` prepares a shorter value for

Azure services with stricter naming limits. The `tags` map gives resources the standard `Project`, `Environment`, `ManagedBy`, and `Lab` labels.

The resource group consumes those values without rebuilding them:

```
resource "azurerm_resource_group" "lab" {
  name     = "${local.prefix}-rg"
  location = var.location
  tags     = local.tags
}
```

This is the reason locals matter. The resource block stays focused on the Azure object. Naming and tag composition stay centralized in `locals.tf`. If the pattern changes, the logic is easier to find and review.

Outputs expose the values that help validate the result:

```
output "resource_group_name" {
  description = "Resource group created by this lab."
  value       = azurerm_resource_group.lab.name
}
output "location" {
  description = "Azure region used by this lab."
  value       = azurerm_resource_group.lab.location
}
```

After apply, `terraform output` should show the resource group name and location. With default values, the expected output is `clz-dev-clz150-rg` and `eastus2`. Outputs are useful because they turn deployed facts into values Terraform can display and share with connected configuration.

The standard command loop remains the same:

```
terraform init
terraform fmt -check
terraform validate
terraform plan -out tfplan
terraform apply tfplan
terraform output
terraform destroy
```

All commands should be run from `CLZ-150-variables-locals-outputs`. The plan review should trace the value flow. The inputs should produce the expected locals, the locals should produce the expected resource group name and tags, and the outputs should match the resource being created.

The main design rule is simple: put a value where it belongs. A value that should vary belongs in a variable. A value computed from other values belongs in a local. A value needed after evaluation or deployment belongs in an output. A safe example belongs in `terraform.tfvars.example`; a real local override belongs outside committed files.

Cleanup follows the same discipline. `terraform destroy` should run from the Chapter 6 folder, and the destroy plan should refer only to `clz-dev-clz150-rg`. Generated local files such as real `terraform.tfvars`, saved plans, and state files should not be committed.

By the end of the chapter, the reader has the value model used by the rest of the book. As resources become more complex, the same structure keeps them readable: variables for input, locals for composition, outputs for validation and handoff. Terraform remains easier to reason about when values have clear jobs instead of being repeated inline.

Chapter 7: State And Locking Basics

This chapter is based on the `CLZ-160-state-and-locking-basics` lab. The lab introduces local Terraform state, runtime file safety, and the habit of destroying resources from the folder that owns them.

Terraform is not only a command runner. It keeps a record of what it manages. That record is state. State connects the resources written in configuration to the real objects Terraform has created or inspected in Azure.

Manual Azure work does not provide that local management record. A person can create a resource group in the portal, but Terraform will not know that the portal-created object belongs to a specific folder unless it is imported or created through Terraform. State is the link that lets Terraform compare configuration, recorded ownership, and deployed infrastructure.

This lab uses local state only. It does not configure remote state storage or a shared backend yet. That choice keeps the lesson focused on the basics: state is important, runtime files are not source files, and cleanup should happen from the same folder that created the resource.

The lab creates one tagged resource group in `eastus2` with a name that starts with `clz-dev-clz160`. The resource group is intentionally simple. It gives Terraform one Azure object to track while the reader focuses on the relationship between code, state, and real infrastructure.

The folder uses the standard lab layout. `versions.tf` declares Terraform and provider constraints. `providers.tf` configures AzureRM. `variables.tf` defines inputs. `locals.tf` builds names and tags. `resource-group.tf` creates the resource group. `lab.tf` records the lesson checkpoint. `outputs.tf` exposes validation values. `terraform.tfvars.example` provides safe example inputs.

The provider still defines the Azure connection:

```
provider "azurerm" {
  features {}
  subscription_id = var.subscription_id
}
```

State does not replace provider authentication. Terraform still needs AzureRM and an Azure subscription context before it can plan or apply. State records what Terraform manages after configuration and provider access are evaluated together.

The lab identity comes from variables:

```
variable "location" {
  description = "Azure region for this lab."
  type        = string
  default     = "eastus2"
}
variable "lab_id" {
  description = "Azure From Zero To Hero lesson ID."
  type        = string
  default     = "CLZ-160"
}
```

```
variable "name_prefix" {
  description = "Short prefix used in Azure resource names."
  type        = string
  default     = "clz"
}
```

Those inputs help the lab produce a recognizable Azure object. Recognizable names matter because state safety is easier when the plan, outputs, and Azure portal all point to the same expected resource.

The safe example values keep the local run predictable:

```
environment     = "dev"
location        = "eastus2"
lab_id          = "CLZ-160"
name_prefix     = "clz"
subscription_id = null
```

If a reader needs local overrides, the example file can be copied to `terraform.tfvars`. The copied file is a local runtime input and should not be committed. The same safety mindset applies to state files, saved plans, and the `.terraform` directory.

The locals create the resource identity:

```
locals {
  lab_number     = replace(var.lab_id, "CLZ-", "")
  lab_code       = lower(replace(var.lab_id, "CLZ-", "clz"))
  prefix         = lower("${var.name_prefix}-${var.environment}-${local.lab_code}")
  compact_prefix = substr(replace(local.prefix, "-", ""), 0, 18)
  tags = {
    Project     = "Azure From Zero To Hero"
    Environment = var.environment
    ManagedBy   = "Terraform"
    Lab         = var.lab_id
  }
}
```

With default values, `local.prefix` becomes `clz-dev-clz160`. The tags identify the project, environment, management method, and lab. The `ManagedBy = "Terraform"` tag is especially useful here because it reinforces that the resource should be managed through Terraform, not casual portal edits.

The managed Azure resource is the resource group:

```
resource "azurerm_resource_group" "lab" {
  name     = "${local.prefix}-rg"
  location = var.location
  tags     = local.tags
}
```

In configuration, this object is addressed as `azurerm_resource_group.lab`. With default values, the Azure resource group name is `clz-dev-clz160-rg`. Terraform state is what connects that configuration address to the real Azure object after creation.

The outputs give the reader a clean validation surface:

```
output "resource_group_name" {
  description = "Resource group created by this lab."
  value       = azurerm_resource_group.lab.name
}
output "location" {
  description = "Azure region used by this lab."
  value       = azurerm_resource_group.lab.location
```

```
}
```

After apply, `terraform output` should show the resource group name and location. With default values, those outputs should be `clz-dev-clz160-rg` and `eastus2`. The outputs confirm what Terraform believes it manages without exposing state file contents.

The lab runbook stays familiar:

```
terraform init
terraform fmt -check
terraform validate
terraform plan -out tfplan
terraform apply tfplan
terraform output
terraform destroy
```

All commands should run from `CLZ-160-state-and-locking-basics`. That folder boundary is important because local state belongs to the working folder unless a backend is configured. Running Terraform from the wrong folder can separate the configuration from the state that belongs with it.

State files should not be treated as normal source files. Local state, backup state, saved plans, real `terraform.tfvars`, and `.terraform` contents are runtime artifacts. They can include environment-specific information, provider selections, resource identifiers, or operational details. The reusable source is the `.tf` configuration and safe examples, not the local runtime record.

Locking protects state from overlapping operations. A lock prevents two Terraform operations from writing to the same state at the same time. This lab does not set up a remote backend, but the safety rule still applies locally: do not run competing applies or destroys against the same working state.

Plan review should remain simple. The plan should show one `azurerm_resource_group.lab`, the name `clz-dev-clz160-rg`, the `eastus2` location, and the standard tags. If the plan shows unrelated resources or an unexpected name, the workflow should stop before apply.

Cleanup is part of state safety. `terraform destroy` should run from the same folder that created the resource, and the destroy plan should point to the Chapter 7 resource group. Deleting the resource manually in the portal may leave Terraform state out of sync with Azure. The safer learning habit is to let Terraform destroy what Terraform created.

By the end of the chapter, state should feel less like a hidden file and more like Terraform's management record. It should be protected, kept out of normal commits, and used from the correct folder. That discipline becomes even more important when state tracks connected Azure resources such as VNets, subnets, security rules, and compute attachments.

Chapter 8: Virtual Network Foundation

This chapter is based on the `CLZ-170-virtual-network-foundation` lab. The lab creates one Azure virtual network and four subnets for web, app, data, and management workloads.

Manual network design is fragile because address choices are easy to scatter across clicks and memory. A VNet can be created with a reasonable range, then later subnets can be added without a clear plan. Names may drift. Address ranges may become harder to explain. Security work becomes harder when the network foundation is not explicit.

Terraform makes the network shape visible before deployment. The VNet address space, subnet names, subnet prefixes, resource group, location, and tags are all in files. The plan can be reviewed before Azure is changed. That makes network design a source-controlled decision instead of a portal history problem.

The lab creates one resource group, one VNet, and four subnets. The VNet address space is `10.40.0.0/16`. The web subnet is `10.40.1.0/24`, the app subnet is `10.40.2.0/24`, the data subnet is `10.40.3.0/24`, and the management subnet is `10.40.10.0/24`. The ranges are intentionally simple so the subnet roles are easy to inspect.

The folder uses the standard lab layout. `versions.tf` declares Terraform and provider constraints. `providers.tf` configures AzureRM. `variables.tf` defines shared inputs. `locals.tf` builds names and tags. `resource-group.tf` creates the resource group. `network.tf` creates the VNet and subnets. `outputs.tf` exposes validation values. `terraform.tfvars.example` provides safe example inputs.

The lab identity comes from safe example values:

```
environment     = "dev"
location        = "eastus2"
lab_id          = "CLZ-170"
name_prefix     = "clz"
subscription_id = null
```

With default values, `local.prefix` becomes `clz-dev-clz170`. The resource group becomes `clz-dev-clz170-rg`, and the virtual network becomes `clz-dev-clz170-vnet`. If local overrides are needed, the copied `terraform.tfvars` file should stay out of commits.

The naming and tag locals carry forward the shared pattern:

```
locals {
  lab_number     = replace(var.lab_id, "CLZ-", "")
  lab_code       = lower(replace(var.lab_id, "CLZ-", "clz"))
  prefix         = lower("${var.name_prefix}-${var.environment}-${local.lab_code}")
  compact_prefix = substr(replace(local.prefix, "-", ""), 0, 18)
  tags = {
    Project     = "Azure From Zero To Hero"
    Environment = var.environment
    ManagedBy   = "Terraform"
    Lab         = var.lab_id
  }
}
```

The VNet uses the same project, environment, management, and lab tags as the rest of the lab model. That consistency matters because network resources often become shared dependencies for compute, security, and private access patterns.

The resource group remains the lab boundary:

```
resource "azurerm_resource_group" "lab" {
  name     = "${local.prefix}-rg"
  location = var.location
  tags     = local.tags
}
```

The virtual network is defined in `network.tf`:

```
resource "azurerm_virtual_network" "main" {
  name                = "${local.prefix}-vnet"
  address_space       = ["10.40.0.0/16"]
  location            = azurerm_resource_group.lab.location
  resource_group_name = azurerm_resource_group.lab.name
  tags                = local.tags
}
```

The VNet is the parent address boundary for the lab. It takes its location and resource group from Terraform references, which avoids manually repeating those values. The address space `10.40.0.0/16` leaves room for separate subnet ranges while keeping the design inside one VNet.

The web subnet is the first subnet:

```
resource "azurerm_subnet" "web" {
  name                 = "web-snet"
  resource_group_name  = azurerm_resource_group.lab.name
  virtual_network_name = azurerm_virtual_network.main.name
  address_prefixes     = ["10.40.1.0/24"]
}
```

The app and data subnets follow the same resource pattern:

```
resource "azurerm_subnet" "app" {
  name                 = "app-snet"
  resource_group_name  = azurerm_resource_group.lab.name
  virtual_network_name = azurerm_virtual_network.main.name
  address_prefixes     = ["10.40.2.0/24"]
}
resource "azurerm_subnet" "data" {
  name                 = "data-snet"
  resource_group_name  = azurerm_resource_group.lab.name
  virtual_network_name = azurerm_virtual_network.main.name
  address_prefixes     = ["10.40.3.0/24"]
}
```

This separation gives security rules a clean target. Web, app, and data workloads can be discussed separately because the network already separates their address spaces.

The management subnet is kept apart from the workload subnets:

```
resource "azurerm_subnet" "mgmt" {
  name                 = "mgmt-snet"
  resource_group_name  = azurerm_resource_group.lab.name
  virtual_network_name = azurerm_virtual_network.main.name
  address_prefixes     = ["10.40.10.0/24"]
}
```

The management subnet reserves address space for administrative access patterns. It is not used to deploy Bastion or management tooling yet. It creates a dedicated network slot where those controls can be attached without redesigning the VNet.

The outputs expose the main validation values:

```
output "resource_group_name" {
  description = "Resource group created by this lab."
  value       = azurerm_resource_group.lab.name
}
output "location" {
  description = "Azure region used by this lab."
  value       = azurerm_resource_group.lab.location
}
output "vnet_name" {
  description = "Virtual network name."
  value       = azurerm_virtual_network.main.name
}
```

After apply, `terraform output` should show the resource group name, location, and VNet name. With default values, the VNet output should be `clz-dev-clz170-vnet`. Subnet names and prefixes should be reviewed in the plan and checked in Azure if the lab is applied.

The standard Terraform runbook still applies:

```
terraform init
terraform fmt -check
terraform validate
terraform plan -out tfplan
terraform apply tfplan
terraform output
terraform destroy
```

All commands should run from `CLZ-170-virtual-network-foundation`. The plan should show one resource group, one virtual network, and four subnets. The resource names should start with `clz-dev-clz170` where the lab naming model applies, and the subnet names should match `web-snet`, `app-snet`, `data-snet`, and `mgmt-snet`.

The most important review point is address accuracy. The VNet should use `10.40.0.0/16`, and the subnet prefixes should be `10.40.1.0/24`, `10.40.2.0/24`, `10.40.3.0/24`, and `10.40.10.0/24`. A wrong range should stop the workflow before apply because network mistakes are harder to unwind after other resources depend on them.

Cleanup should be performed from the same folder. `terraform destroy` should remove the Chapter 8 resource group, VNet, and subnets. After cleanup, generated local files such as real `terraform.tfvars`, saved plans, and state files should not be committed.

By the end of the chapter, the reader has moved from single-resource labs into a real network foundation. Terraform now tracks connected resources, not just an isolated resource group. Once the foundation exists, subnet traffic rules can be made reviewable in code through Network Security Groups and explicit associations.

Chapter 9: Network Security Groups

This chapter is based on the `CLZ-180-network-security-groups` lab. The lab creates a web subnet Network Security Group, adds inbound HTTP and scoped RDP rules, and associates the NSG with the web subnet.

Manual firewall rules are easy to create and hard to explain later. A rule can be added in the Azure portal with a few clicks, but the source range, priority, direction, protocol, port, and target may not be obvious during review. Security rules should not depend on memory. They should be visible before they change the network.

Terraform turns those choices into reviewable code. The NSG is a resource. Each rule is a resource. The subnet association is a resource. The plan shows what traffic is being allowed, where it is allowed from, which priority is used, and which subnet receives the rule set.

The lab builds on the virtual network foundation. It creates one resource group, one VNet, four subnets, one web NSG, two inbound rules, and one association to `web-snet`. The NSG is attached to the web subnet only. It does not automatically apply to every subnet in the VNet.

The folder uses the standard lab layout. `versions.tf` declares Terraform and provider constraints. `providers.tf` configures AzureRM. `variables.tf` defines shared inputs, including `admin_cidr`. `locals.tf` builds names and tags. `resource-group.tf` creates the resource group. `security.tf` creates the network and NSG resources. `outputs.tf` exposes validation values. `terraform.tfvars.example` provides safe example inputs.

The safe example values include the scoped admin source:

```
environment     = "dev"
location        = "eastus2"
lab_id          = "CLZ-180"
name_prefix     = "clz"
admin_cidr      = "203.0.113.10/32"
subscription_id = null
```

The `admin_cidr` value is a documentation-safe example range. A real administrator source address belongs in local input, not in committed source. If the example file is copied to `terraform.tfvars`, the copied file should stay local.

The shared locals create the naming base:

```
locals {
  lab_number     = replace(var.lab_id, "CLZ-", "")
  lab_code       = lower(replace(var.lab_id, "CLZ-", "clz"))
  prefix         = lower("${var.name_prefix}-${var.environment}-${local.lab_code}")
  compact_prefix = substr(replace(local.prefix, "-", ""), 0, 18)
  tags = {
    Project     = "Azure From Zero To Hero"
    Environment = var.environment
    ManagedBy   = "Terraform"
    Lab         = var.lab_id
  }
}
```

With default values, `local.prefix` becomes `clz-dev-clz180`. The web NSG name becomes `clz-dev-clz180-web-nsg`. The tags identify the project, environment, Terraform ownership, and lab.

The network target is defined first:

```
resource "azurerm_virtual_network" "main" {
  name                = "${local.prefix}-vnet"
  address_space       = ["10.40.0.0/16"]
  location            = azurerm_resource_group.lab.location
  resource_group_name = azurerm_resource_group.lab.name
  tags                = local.tags
}
resource "azurerm_subnet" "web" {
  name                 = "web-snet"
  resource_group_name  = azurerm_resource_group.lab.name
  virtual_network_name = azurerm_virtual_network.main.name
  address_prefixes     = ["10.40.1.0/24"]
}
```

The lab also defines app, data, and management subnets, but the NSG association in this chapter targets `web-snet`. That focus matters. A security rule is not complete until its target is clear.

The NSG resource holds the rule set:

```
resource "azurerm_network_security_group" "web" {
  name                = "${local.prefix}-web-nsg"
  location            = azurerm_resource_group.lab.location
  resource_group_name = azurerm_resource_group.lab.name
  tags                = local.tags
}
```

The NSG name identifies its purpose. It is not a general network NSG. It is the web subnet NSG for this lab.

The HTTP rule allows inbound web traffic:

```
resource "azurerm_network_security_rule" "allow_http" {
  name                        = "allow-http"
  priority                    = 100
  direction                   = "Inbound"
  access                      = "Allow"
  protocol                    = "Tcp"
  source_port_range           = "*"
  destination_port_range      = "80"
  source_address_prefix       = "Internet"
  destination_address_prefix  = "*"
  resource_group_name         = azurerm_resource_group.lab.name
  network_security_group_name = azurerm_network_security_group.web.name
}
```

This rule allows TCP traffic from `Internet` to destination port `80`. The priority is `100`, so it is evaluated before the later RDP rule at priority `110`. The rule is broad by source because HTTP is the public web path in this example.

The RDP rule is intentionally scoped:

```
resource "azurerm_network_security_rule" "allow_admin_rdp" {
  name                        = "allow-admin-rdp"
  priority                    = 110
  direction                   = "Inbound"
  access                      = "Allow"
  protocol                    = "Tcp"
  source_port_range           = "*"
  destination_port_range      = "3389"
```

```
  source_address_prefix       = var.admin_cidr
  destination_address_prefix  = "*"
  resource_group_name         = azurerm_resource_group.lab.name
  network_security_group_name = azurerm_network_security_group.web.name
}
```

This rule allows TCP traffic to destination port `3389`, but only from `var.admin_cidr`. It should not be rewritten as broad public RDP. Stronger private access patterns can replace this public lab access model, but this lab's immediate lesson is that administrative access must be constrained and visible.

The association applies the NSG to the web subnet:

```
resource "azurerm_subnet_network_security_group_association" "web" {
  subnet_id                 = azurerm_subnet.web.id
  network_security_group_id = azurerm_network_security_group.web.id
}
```

This association is the binding step. Without it, the NSG and rules would exist as Azure objects but would not protect `web-snet`.

The outputs expose the basic validation values:

```
output "resource_group_name" {
  description = "Resource group created by this lab."
  value       = azurerm_resource_group.lab.name
}
output "location" {
  description = "Azure region used by this lab."
  value       = azurerm_resource_group.lab.location
}
output "web_nsg_name" {
  description = "Web subnet NSG name."
  value       = azurerm_network_security_group.web.name
}
```

After apply, `terraform output` should show the resource group, location, and web NSG name. With default values, the NSG output should be `clz-dev-clz180-web-nsg`.

The standard Terraform runbook still applies:

```
terraform init
terraform fmt -check
terraform validate
terraform plan -out tfplan
terraform apply tfplan
terraform output
terraform destroy
```

All commands should run from `CLZ-180-network-security-groups`. The plan should show the network foundation, the web NSG, `allow-http`, `allow-admin-rdp`, and the web subnet association. The review should verify priorities, directions, access values, protocols, ports, source ranges, and the subnet target.

The most important review point is least access. HTTP is open from `Internet` because it represents public web traffic. RDP is not open from `Internet`; it is limited to `var.admin_cidr`. A plan that broadens RDP access should stop the workflow before apply.

Cleanup should run from the same folder. `terraform destroy` should remove the Chapter 9 resource group, VNet, subnets, NSG, rules, and association. Generated local files such as real `terraform.tfvars`, saved plans, and state files should not be committed.

By the end of the chapter, subnet security is no longer a click-built setting. It is a reviewable part of the Azure design. That foundation is ready to host compute because VM placement and access rules are already visible in Terraform.

Chapter 10: Windows VM Basics

This chapter is based on the `CLZ-190-windows-vm-basics` lab. The lab creates the first Windows Server virtual machine with generated administrator credentials, a public IP, a network interface, and sensitive output handling.

Manual VM builds hide many decisions behind a portal form. Image selection, VM size, disk type, subnet placement, public IP configuration, administrator username, and password handling all affect the final server. If those decisions are clicked once and not captured, the next server may not match the first. Terraform makes the VM build reviewable before Azure creates it.

The lab includes a complete network and NSG baseline: resource group, VNet, four subnets, web NSG, HTTP rule, scoped RDP rule, and web subnet association. It then adds the Windows compute layer: a generated password, Standard public IP, network interface, and Windows Server VM.

The VM is not yet configured as a web server. This chapter focuses on the base compute pattern: how Terraform assembles the network, credential, public endpoint, NIC, OS image, VM size, disk, and outputs into one repeatable Windows host.

The folder uses the standard lab layout. `versions.tf` declares Terraform and provider constraints. `providers.tf` configures AzureRM. `variables.tf` defines shared inputs, including `admin_username` and `admin_cidr`. `locals.tf` builds names and tags. `resource-group.tf` creates the resource group. `compute-windows.tf` creates the network, security, credential, public IP, NIC, and VM resources. `outputs.tf` exposes validation values. `terraform.tfvars.example` provides safe example inputs.

The safe example values include the lab identity and administrator username:

```
environment     = "dev"
location        = "eastus2"
lab_id          = "CLZ-190"
name_prefix     = "clz"
admin_username  = "clzadmin"
admin_cidr      = "203.0.113.10/32"
subscription_id = null
```

The example file does not store a real password. Terraform generates the administrator password during the run. If local overrides are needed, the copied `terraform.tfvars` file should stay out of commits.

The shared locals create the naming base:

```
locals {
  lab_number     = replace(var.lab_id, "CLZ-", "")
  lab_code       = lower(replace(var.lab_id, "CLZ-", "clz"))
  prefix         = lower("${var.name_prefix}-${var.environment}-${local.lab_code}")
  compact_prefix = substr(replace(local.prefix, "-", ""), 0, 18)
  tags = {
    Project     = "Azure From Zero To Hero"
```

```
    Environment = var.environment
    ManagedBy   = "Terraform"
    Lab         = var.lab_id
  }
}
```

With default values, `local.prefix` becomes `clz-dev-clz190`. The public IP becomes `clz-dev-clz190-web-pip`, the NIC becomes `clz-dev-clz190-web-nic`, and the VM becomes `clz-dev-clz190-web-winvm`. The VM computer name is built from the lab number as `clz190`.

The generated password resource is the credential source:

```
resource "random_password" "windows_admin" {
  length           = 20
  special          = true
  min_upper        = 2
  min_lower        = 2
  min_numeric      = 2
  min_special      = 2
  override_special = "!#$%&*()-_=+[]{}<>:?"
}
```

This avoids a committed password. The generated value is passed into the Windows VM and exposed only through a sensitive output. That does not make the value unimportant; it means the value must be handled deliberately if retrieved locally.

The public IP provides the first-access endpoint:

```
resource "azurerm_public_ip" "web" {
  name                = "${local.prefix}-web-pip"
  location            = azurerm_resource_group.lab.location
  resource_group_name = azurerm_resource_group.lab.name
  allocation_method   = "Static"
  sku                 = "Standard"
  tags                = local.tags
}
```

The lab uses a Static Standard public IP so the VM has a stable access point for validation. This is a lab access path, not a blanket recommendation that all future administration should happen through public exposure.

The network interface connects the VM to the web subnet:

```
resource "azurerm_network_interface" "web" {
  name                = "${local.prefix}-web-nic"
  location            = azurerm_resource_group.lab.location
  resource_group_name = azurerm_resource_group.lab.name
  ip_configuration {
    name                          = "internal"
    subnet_id                     = azurerm_subnet.web.id
    private_ip_address_allocation = "Dynamic"
    public_ip_address_id          = azurerm_public_ip.web.id
  }
  tags = local.tags
}
```

The NIC references the web subnet and public IP directly. Terraform uses those references to understand the dependency order. The VM cannot be connected correctly until the subnet, public IP, and NIC exist.

The Windows VM resource ties the compute configuration together:

```
resource "azurerm_windows_virtual_machine" "web" {
  name                = "${local.prefix}-web-winvm"
```

```
  computer_name       = "clz${local.lab_number}"
  location            = azurerm_resource_group.lab.location
  resource_group_name = azurerm_resource_group.lab.name
  size                = "Standard_B2s"
  admin_username      = var.admin_username
  admin_password      = random_password.windows_admin.result
  network_interface_ids = [
    azurerm_network_interface.web.id
  ]
  os_disk {
    caching              = "ReadWrite"
    storage_account_type = "Standard_LRS"
  }
  source_image_reference {
    publisher = "MicrosoftWindowsServer"
    offer     = "WindowsServer"
    sku       = "2022-datacenter-azure-edition"
    version   = "latest"
  }
  tags = local.tags
}
```

The VM uses `Standard_B2s`, a Standard_LRS OS disk, and the Windows Server `2022-datacenter-azure-edition` image. It uses `var.admin_username` for the administrator username and `random_password.windows_admin.result` for the password. The NIC reference connects the VM to the web subnet and public IP path.

The outputs expose the validation values that matter:

```
output "windows_vm_public_ip" {
  description = "Public IP for first-access validation."
  value       = azurerm_public_ip.web.ip_address
}
output "windows_admin_password" {
  description = "Generated Windows admin password."
  value       = random_password.windows_admin.result
  sensitive   = true
}
```

The public IP output supports first-access validation after deployment. The password output is marked `sensitive = true`, so Terraform treats it differently from ordinary output. If it is retrieved locally, it should be handled as a secret and never committed, pasted into the manuscript, or stored in casual notes.

The standard Terraform runbook still applies:

```
terraform init
terraform fmt -check
terraform validate
terraform plan -out tfplan
terraform apply tfplan
terraform output
terraform destroy
```

All commands should run from `CLZ-190-windows-vm-basics`. The plan should show the network foundation, web NSG, scoped RDP rule, generated password, public IP, NIC, and Windows VM. It should not contain a hard-coded administrator password.

The review should confirm the VM name, computer name, VM size, Windows image, OS disk type, NIC attachment, public IP output, and sensitive password output. It should also confirm

that RDP remains scoped through `admin_cidr`. A change that broadens administrative access should stop the workflow before apply.

Cleanup should run from the same folder. `terraform destroy` should remove the Chapter 10 resource group and the resources inside it. Generated local files such as real `terraform.tfvars`, saved plans, state files, and any locally retrieved password values should not be committed.

By the end of this chapter, the reader has the first repeatable Windows VM pattern. Terraform now manages compute, network, security, credentials, and outputs together. With the host pattern defined, guest automation can be added deliberately instead of being mixed into the first compute lesson.

Chapter 11: Windows VM IIS Bootstrap

This chapter is based on the `CLZ-200-windows-vm-iis-bootstrap` lab. The lab creates a Windows Server web node with IIS and a custom validation page.

A repeatable Windows VM is useful only when the server role is repeatable too. A VM that is built by Terraform but configured by hand is still only partly repeatable. If IIS is installed through an RDP session, the build depends on memory, timing, and whoever performed the manual steps. The infrastructure may be in code, but the server role is not.

CLZ-200 solves that first configuration gap with Azure's Custom Script Extension. The lab keeps the Windows VM, public IP, NIC, generated administrator password, web subnet, HTTP rule, scoped RDP rule, and standard tags from the base compute pattern. Then it adds `azurerm_virtual_machine_extension.iis`, which runs a PowerShell command inside the guest operating system after Azure provisions the VM.

The folder layout is familiar. `versions.tf` requires Terraform `>= 1.6.0` and declares the AzureRM and Random providers. `providers.tf` configures AzureRM with `subscription_id = var.subscription_id`. `variables.tf` defines the shared curriculum inputs. `locals.tf` builds `local.prefix`, `local.compact_prefix`, and the standard tags. `resource-group.tf` creates the lab resource group. `compute-windows.tf` creates the network, NSG rules, generated password, public IP, NIC, VM, and IIS extension. `outputs.tf` exposes validation values.

The safe example values identify this lab:

```
environment     = "dev"
location        = "eastus2"
lab_id          = "CLZ-200"
name_prefix     = "clz"
admin_username  = "clzadmin"
admin_cidr      = "203.0.113.10/32"
instance_count  = 2
subscription_id = null
```

With those defaults, `local.prefix` becomes `clz-dev-clz200`. The resource group becomes `clz-dev-clz200-rg`, the VNet becomes `clz-dev-clz200-vnet`, the web NSG becomes `clz-dev-clz200-web-nsg`, the public IP becomes `clz-dev-clz200-web-pip`, the NIC becomes `clz-dev-clz200-web-nic`, and the VM becomes `clz-dev-clz200-web-winvm`. The VM computer name is `clz200`.

The VM resource still defines the operating system host:

```
resource "azurerm_windows_virtual_machine" "web" {
  name                = "${local.prefix}-web-winvm"
  computer_name       = "clz${local.lab_number}"
  location            = azurerm_resource_group.lab.location
  resource_group_name = azurerm_resource_group.lab.name
  size                = "Standard_B2s"
  admin_username      = var.admin_username
  admin_password      = random_password.windows_admin.result
  network_interface_ids = [
    azurerm_network_interface.web.id
```

```
  ]
  os_disk {
    caching              = "ReadWrite"
    storage_account_type = "Standard_LRS"
  }
  source_image_reference {
    publisher = "MicrosoftWindowsServer"
    offer     = "WindowsServer"
    sku       = "2022-datacenter-azure-edition"
    version   = "latest"
  }
  tags = local.tags
}
```

The VM uses `Standard_B2s`, a `Standard_LRS` OS disk, and the Windows Server `2022-datacenter-azure-edition` image. Its administrator password comes from `random_password.windows_admin.result`, so the password is generated during deployment rather than stored in source.

The IIS bootstrap happens in the extension resource:

```
resource "azurerm_virtual_machine_extension" "iis" {
  name                       = "install-iis"
  virtual_machine_id         = azurerm_windows_virtual_machine.web.id
  publisher                  = "Microsoft.Compute"
  type                       = "CustomScriptExtension"
  type_handler_version       = "1.10"
  auto_upgrade_minor_version = true
  settings = jsonencode({
    commandToExecute = "powershell -ExecutionPolicy Unrestricted -Command \"Install-WindowsFeature -Name Web-Server -IncludeManagementTools; Set-Content -Path C:\\inetpub\\wwwroot\\index.html -Value 'Azure From Zero To Hero IIS ${local.prefix}'\""
  })
}
```

The dependency is explicit through `virtual_machine_id = azurerm_windows_virtual_machine.web.id`. Terraform cannot create the extension until the VM exists. The extension uses the `Microsoft.Compute` publisher and `CustomScriptExtension` type with handler version `1.10`.

The PowerShell command installs IIS with `Install-WindowsFeature -Name Web-Server -IncludeManagementTools`. It then writes `C:\inetpub\wwwroot\index.html` with the text `Azure From Zero To Hero IIS ${local.prefix}`. That small page is the validation target. It proves that the VM was created, the extension ran, IIS is serving traffic, and the response belongs to this lab.

HTTP access depends on the NSG rule from the same file:

```
resource "azurerm_network_security_rule" "allow_http" {
  name                        = "allow-http"
  priority                    = 100
  direction                   = "Inbound"
  access                      = "Allow"
  protocol                    = "Tcp"
  source_port_range           = "*"
  destination_port_range      = "80"
  source_address_prefix       = "Internet"
  destination_address_prefix  = "*"
  resource_group_name         = azurerm_resource_group.lab.name
  network_security_group_name = azurerm_network_security_group.web.name
```

```
}
```

RDP remains scoped separately through `admin_cidr`. The chapter uses direct RDP only as a lab access pattern. Do not broaden administrative access to make troubleshooting easier; fix the configuration or the bootstrap instead.

The IIS URL output provides the web validation endpoint:

```
output "iis_url" {
  description = "IIS validation URL."
  value       = "http://${azurerm_public_ip.web.ip_address}"
}
```

The administrator password remains a sensitive output:

```
output "windows_admin_password" {
  description = "Generated Windows admin password."
  value       = random_password.windows_admin.result
  sensitive   = true
}
```

Run the lab from `CLZ-200-windows-vm-iis-bootstrap`:

```
terraform init
terraform fmt -check
terraform validate
terraform plan -out tfplan
terraform apply tfplan
terraform output
terraform destroy
```

Before apply, the plan should include the base VM resources and one new extension: `azurerm_virtual_machine_extension.iis`. Confirm the extension name is `install-iis`, the type is `CustomScriptExtension`, the target is the Windows VM, and the command installs `Web-Server`. Also confirm that `iis_url` is present and that `windows_admin_password` remains sensitive.

After apply, request the web endpoint:

```
$iisUrl = terraform output -raw iis_url
Invoke-WebRequest -UseBasicParsing $iisUrl
```

The response should include `Azure From Zero To Hero IIS clz-dev-clz200`. If validation fails, separate the problem into layers. If the request times out, check the public IP, HTTP rule, subnet association, and VM reachability. If IIS responds with the wrong content, check the extension status and the command text. If the VM is created but the extension fails, the infrastructure build is not complete for this chapter.

Cleanup follows the same lab ownership discipline. Run `terraform destroy` from the lab folder. Do not commit copied `terraform.tfvars`, saved plans, state files, or locally retrieved passwords. The generated administrator password should stay out of the manuscript and out of casual notes. By the end of this chapter, the VM is a repeatable Windows web node instead of an empty host. Terraform now controls both the Azure infrastructure and the first server bootstrap step. That keeps application configuration in the deployment path and lets administrative access remain a separate design decision.

Chapter 12: Azure Bastion RDP

This chapter is based on the `CLZ-210-azure-bastion-rdp` lab. The lab creates a private Windows VM reachable through Azure Bastion.

The first Windows chapters used public reachability to make VM and IIS validation straightforward. Administration should move to a different model. RDP is a management protocol, and exposing it through a VM public IP creates a path that must be defended even when the source CIDR is scoped. As the environment becomes more complete, direct public VM administration should be removed.

CLZ-210 changes the management path. The Windows VM is still created by Terraform, but the VM network interface is private only. Azure Bastion receives the public IP and provides browser-based RDP into the virtual network. The connection path becomes Azure portal to Bastion to private VM, rather than internet to VM public IP.

The folder follows the standard lab layout. `versions.tf` requires Terraform `>= 1.6.0` and declares the AzureRM and Random providers. `providers.tf` configures AzureRM with `subscription_id = var.subscription_id`. `variables.tf` defines shared inputs. `locals.tf` builds `local.prefix`, `local.compact_prefix`, and the standard tags. `resource-group.tf` creates the lab resource group. `bastion.tf` creates the network, security rules, generated password, Bastion subnet, Bastion public IP, Bastion host, private NIC, and private Windows VM. `outputs.tf` exposes validation values.

The safe example values identify this lab:

```
environment     = "dev"
location        = "eastus2"
lab_id          = "CLZ-210"
name_prefix     = "clz"
admin_username  = "clzadmin"
admin_cidr      = "203.0.113.10/32"
instance_count  = 2
subscription_id = null
```

With those defaults, `local.prefix` becomes `clz-dev-clz210`. Expected names include `clz-dev-clz210-rg`, `clz-dev-clz210-vnet`, `clz-dev-clz210-bas-pip`, `clz-dev-clz210-bastion`, `clz-dev-clz210-private-web-nic`, and `clz-dev-clz210-private-web-winvm`. The VM computer name is `clz210`.

Azure Bastion requires a subnet with a specific name:

```
resource "azurerm_subnet" "bastion" {
  name                 = "AzureBastionSubnet"
  resource_group_name  = azurerm_resource_group.lab.name
  virtual_network_name = azurerm_virtual_network.main.name
  address_prefixes     = ["10.40.20.0/26"]
}
```

The name must be exactly `AzureBastionSubnet`. The lab uses `10.40.20.0/26`, keeping Bastion separate from `web-snet`, `app-snet`, `data-snet`, and `mgmt-snet`. During review, both the exact subnet name and the separate address range matter.

Bastion gets the public IP:

```
resource "azurerm_public_ip" "bastion" {
  name                = "${local.prefix}-bas-pip"
  location            = azurerm_resource_group.lab.location
  resource_group_name = azurerm_resource_group.lab.name
  allocation_method   = "Static"
  sku                 = "Standard"
  tags                = local.tags
}
```

This is the public endpoint for the management service, not for the Windows VM. The IP is Static and Standard because Bastion requires a supported public IP configuration.

The Bastion host binds that public IP to the Bastion subnet:

```
resource "azurerm_bastion_host" "main" {
  name                = "${local.prefix}-bastion"
  location            = azurerm_resource_group.lab.location
  resource_group_name = azurerm_resource_group.lab.name
  tags                = local.tags
  ip_configuration {
    name                 = "configuration"
    subnet_id            = azurerm_subnet.bastion.id
    public_ip_address_id = azurerm_public_ip.bastion.id
  }
}
```

The private VM uses a NIC with no public IP:

```
resource "azurerm_network_interface" "private_web" {
  name                = "${local.prefix}-private-web-nic"
  location            = azurerm_resource_group.lab.location
  resource_group_name = azurerm_resource_group.lab.name
  ip_configuration {
    name                          = "internal"
    subnet_id                     = azurerm_subnet.web.id
    private_ip_address_allocation = "Dynamic"
  }
  tags = local.tags
}
```

The absence of `public_ip_address_id` is intentional. The NIC receives a private address from `web-snet`, and Bastion provides the administrative path. A public IP on the Bastion host is expected. A public IP on the VM NIC would weaken the point of this chapter.

The private Windows VM attaches to that NIC:

```
resource "azurerm_windows_virtual_machine" "private_web" {
  name                = "${local.prefix}-private-web-winvm"
  computer_name       = "clz${local.lab_number}"
  location            = azurerm_resource_group.lab.location
  resource_group_name = azurerm_resource_group.lab.name
  size                = "Standard_B2s"
  admin_username      = var.admin_username
  admin_password      = random_password.windows_admin.result
  network_interface_ids = [
    azurerm_network_interface.private_web.id
  ]
  os_disk {
    caching              = "ReadWrite"
    storage_account_type = "Standard_LRS"
```

```
  }
  source_image_reference {
    publisher = "MicrosoftWindowsServer"
    offer     = "WindowsServer"
    sku       = "2022-datacenter-azure-edition"
    version   = "latest"
  }
  tags = local.tags
}
```

The VM still uses `Standard_B2s`, a `Standard_LRS` OS disk, and the Windows Server `2022-datacenter-azure-edition` image. The administrator password is generated by Terraform through the Random provider and remains sensitive in outputs.

The outputs identify the Bastion host and private VM:

```
output "bastion_name" {
  description = "Azure Bastion name."
  value       = azurerm_bastion_host.main.name
}
output "private_windows_vm_id" {
  description = "Private Windows VM ID."
  value       = azurerm_windows_virtual_machine.private_web.id
}
```

The generated password output stays protected:

```
output "windows_admin_password" {
  description = "Generated Windows admin password."
  value       = random_password.windows_admin.result
  sensitive   = true
}
```

Run the lab from `CLZ-210-azure-bastion-rdp`:

```
terraform init
terraform fmt -check
terraform validate
terraform plan -out tfplan
terraform apply tfplan
terraform output
terraform destroy
```

Before apply, review the plan for the access boundary. It should create `AzureBastionSubnet`, `azurerm_public_ip.bastion`, `azurerm_bastion_host.main`, `azurerm_network_interface.private_web`, and `azurerm_windows_virtual_machine.private_web`. It should not attach a VM public IP to the private NIC. It should also keep `windows_admin_password` sensitive.

After apply, validate through the Azure portal by opening the private Windows VM and choosing the Bastion connection option. Use the administrator username and generated password only as needed for that validation. The connection should succeed without assigning a public IP to the VM. If the connection fails, check the Bastion host, required subnet name, VNet placement, private VM ID, and credentials. Do not add a VM public IP as a shortcut.

Cleanup uses `terraform destroy` from the lab folder. Keep copied `terraform.tfvars`, saved plans, state files, and retrieved passwords out of source control. The management model is only useful if the cleanup and secret-handling discipline stays intact.

By the end of this chapter, Windows administration has moved behind a managed Azure access layer. The environment now has a private administration pattern, while application traffic can remain a separate load-balancing concern.

Chapter 13: Standard Load Balancer Windows

This chapter is based on the `CLZ-220-standard-load-balancer-windows` lab. The lab creates a public load balancer with two IIS Windows backends.

A single Windows web node proves that Terraform can build and bootstrap a server. It does not solve traffic distribution. In a manual build, a second VM is often added by copying settings from the first, then clicking through load balancer screens until traffic works. That hides the design. The frontend IP, backend pool, probe, load rule, NIC associations, and backend server count should be reviewable before anything is deployed.

CLZ-220 makes that design explicit. Terraform creates a public Standard Load Balancer with a Static Standard public IP, one backend pool, one TCP probe on port `80`, one load balancing rule, two backend NICs, two Windows VMs, and two IIS bootstrap extensions. The public endpoint belongs to the load balancer. The backend VMs sit behind it and receive traffic through backend pool membership.

The folder follows the standard lab layout. `versions.tf` requires Terraform `>= 1.6.0` and declares the AzureRM and Random providers. `providers.tf` configures AzureRM with `subscription_id = var.subscription_id`. `variables.tf` defines shared inputs. `locals.tf` builds `local.prefix`, `local.compact_prefix`, and the standard tags. `resource-group.tf` creates the lab resource group. `load-balancer.tf` creates the network, NSG rules, load balancer, backend pool, probe, rule, NICs, pool associations, Windows VMs, and IIS extensions. `outputs.tf` exposes validation values.

The safe example values identify this lab:

```
environment     = "dev"
location        = "eastus2"
lab_id          = "CLZ-220"
name_prefix     = "clz"
admin_username  = "clzadmin"
admin_cidr      = "203.0.113.10/32"
instance_count  = 2
subscription_id = null
```

With those defaults, `local.prefix` becomes `clz-dev-clz220`. The resource group becomes `clz-dev-clz220-rg`, the load balancer public IP becomes `clz-dev-clz220-lb-pip`, the load balancer becomes `clz-dev-clz220-web-lb`, the backend NICs become `clz-dev-clz220-web-1-nic` and `clz-dev-clz220-web-2-nic`, and the VMs become `clz-dev-clz220-web-1-winvm` and `clz-dev-clz220-web-2-winvm`.

The frontend starts with a Static Standard public IP:

```
resource "azurerm_public_ip" "lb" {
  name                = "${local.prefix}-lb-pip"
  location            = azurerm_resource_group.lab.location
  resource_group_name = azurerm_resource_group.lab.name
  allocation_method   = "Static"
  sku                 = "Standard"
```

```
  tags                = local.tags
}
```

The Standard Load Balancer uses that IP in a frontend configuration named `public`:

```
resource "azurerm_lb" "web" {
  name                = "${local.prefix}-web-lb"
  location            = azurerm_resource_group.lab.location
  resource_group_name = azurerm_resource_group.lab.name
  sku                 = "Standard"
  tags                = local.tags
  frontend_ip_configuration {
    name                 = "public"
    public_ip_address_id = azurerm_public_ip.lb.id
  }
}
```

The backend pool defines where the rule can send traffic:

```
resource "azurerm_lb_backend_address_pool" "web" {
  name            = "web-backend-pool"
  loadbalancer_id = azurerm_lb.web.id
}
```

The probe checks backend reachability on TCP port `80`:

```
resource "azurerm_lb_probe" "http" {
  name            = "http-probe"
  loadbalancer_id = azurerm_lb.web.id
  protocol        = "Tcp"
  port            = 80
}
```

The load balancing rule connects the public frontend to the backend pool:

```
resource "azurerm_lb_rule" "http" {
  name                           = "http-rule"
  loadbalancer_id                = azurerm_lb.web.id
  protocol                       = "Tcp"
  frontend_port                  = 80
  backend_port                   = 80
  frontend_ip_configuration_name = "public"
  backend_address_pool_ids       = [azurerm_lb_backend_address_pool.web.id]
  probe_id                       = azurerm_lb_probe.http.id
}
```

The rule is the contract for HTTP traffic. It listens on frontend port `80`, forwards to backend port `80`, targets `web-backend-pool`, and uses `http-probe` to decide which backends can receive traffic.

The backend NICs are private NICs in the web subnet:

```
resource "azurerm_network_interface" "web" {
  count               = 2
  name                = "${local.prefix}-web-${count.index + 1}-nic"
  location            = azurerm_resource_group.lab.location
  resource_group_name = azurerm_resource_group.lab.name
  ip_configuration {
    name                          = "internal"
    subnet_id                     = azurerm_subnet.web.id
    private_ip_address_allocation = "Dynamic"
  }
  tags = local.tags
}
```

The backend pool associations attach each NIC to the pool:

```
resource "azurerm_network_interface_backend_address_pool_association" "web" {
  count                   = 2
  network_interface_id    = azurerm_network_interface.web[count.index].id
  ip_configuration_name   = "internal"
  backend_address_pool_id = azurerm_lb_backend_address_pool.web.id
```

```
}
```

This resource is easy to miss in manual builds. Creating two VMs is not enough; their NIC IP configurations must be members of the backend pool.

The two Windows VMs are created with `count = 2`:

```
resource "azurerm_windows_virtual_machine" "web" {
  count               = 2
  name                = "${local.prefix}-web-${count.index + 1}-winvm"
  computer_name       = "clz${local.lab_number}${count.index + 1}"
  location            = azurerm_resource_group.lab.location
  resource_group_name = azurerm_resource_group.lab.name
  size                = "Standard_B2s"
  admin_username      = var.admin_username
  admin_password      = random_password.windows_admin.result
  network_interface_ids = [
    azurerm_network_interface.web[count.index].id
  ]
  os_disk {
    caching              = "ReadWrite"
    storage_account_type = "Standard_LRS"
  }
  source_image_reference {
    publisher = "MicrosoftWindowsServer"
    offer     = "WindowsServer"
    sku       = "2022-datacenter-azure-edition"
    version   = "latest"
  }
  tags = local.tags
}
```

The computer names become `clz2201` and `clz2202`. Both VMs use `Standard_B2s`, a `Standard_LRS` OS disk, and the Windows Server `2022-datacenter-azure-edition` image. The administrator password is generated once and marked sensitive in output.

Each backend receives IIS through a counted Custom Script Extension:

```
resource "azurerm_virtual_machine_extension" "iis" {
  count                      = 2
  name                       = "install-iis"
  virtual_machine_id         = azurerm_windows_virtual_machine.web[count.index].id
  publisher                  = "Microsoft.Compute"
  type                       = "CustomScriptExtension"
  type_handler_version       = "1.10"
  auto_upgrade_minor_version = true
  settings = jsonencode({
    commandToExecute = "powershell -ExecutionPolicy Unrestricted -Command \"Install-
WindowsFeature -Name Web-Server -IncludeManagementTools; Set-Content -Path
C:\\inetpub\\wwwroot\\index.html -Value 'Azure From Zero To Hero backend ${count.index +
1}'\""
  })
}
```

The validation page includes the backend number. That makes the response useful during testing because it shows which backend answered the request.

The load balancer URL output is the main validation value:

```
output "load_balancer_url" {
  description = "Load balancer validation URL."
  value       = "http://${azurerm_public_ip.lb.ip_address}"
}
```

The generated password output remains sensitive:

```
output "windows_admin_password" {
  description = "Generated Windows admin password."
```

```
  value       = random_password.windows_admin.result
  sensitive   = true
}
```

Run the lab from `CLZ-220-standard-load-balancer-windows`:

```
terraform init
terraform fmt -check
terraform validate
terraform plan -out tfplan
terraform apply tfplan
terraform output
terraform destroy
```

Before apply, the plan should show the full load balancer path: Static Standard public IP, Standard Load Balancer, frontend configuration `public`, backend pool `web-backend-pool`, TCP probe `http-probe` on port `80`, rule `http-rule`, two NICs, two backend pool associations, two Windows VMs, and two IIS extensions. It should also keep `windows_admin_password` sensitive.

After apply, request the load balancer endpoint:

```
$lbUrl = terraform output -raw load_balancer_url
Invoke-WebRequest -UseBasicParsing $lbUrl
```

The response should include either `Azure From Zero To Hero backend 1` or `Azure From Zero To Hero backend 2`. If validation fails, follow the request path in order: public IP, load balancer frontend, rule, probe, backend pool association, NSG HTTP rule, VM state, and IIS extension status. A healthy VM that is not associated with the backend pool will not receive load-balanced traffic.

Cleanup uses `terraform destroy` from the lab folder. Do not commit copied `terraform.tfvars`, saved plans, state files, or retrieved password values. The public endpoint in this chapter is for the shared web frontend, not for direct administration of the backend machines.

By the end of this chapter, HTTP traffic reaches a shared frontend and can be distributed across two Windows IIS backends. The design can serve a public web endpoint while keeping direct backend administration separate from the application frontend.

Chapter 14: Load Balancer NAT Rules

This chapter is based on the `CLZ-230-load-balancer-nat-rules` lab. The lab creates a load balancer with HTTP delivery and one scoped RDP NAT rule.

A shared load balancer frontend is the right direction for user traffic, but lab operations sometimes need a direct route to one backend instance. The risky manual shortcut is to add a public IP to a backend VM or open a broad management rule. CLZ-230 keeps the public edge on the Standard Load Balancer and adds one explicit NAT rule for a controlled RDP path to the first backend.

The folder follows the established load balancer structure. `load-balancer-nat.tf` creates the VNet, subnets, web NSG, HTTP rule, scoped RDP rule, Standard public IP, Standard Load Balancer, backend pool, probe, load rule, two backend NICs, two backend pool associations, two Windows VMs, two IIS extensions, one inbound NAT rule, and one NAT rule association. `outputs.tf` exposes the public IP, NAT port, resource group, location, and sensitive password output.

The safe example values identify this lab:

```
environment     = "dev"
location        = "eastus2"
lab_id          = "CLZ-230"
name_prefix     = "clz"
admin_username  = "clzadmin"
admin_cidr      = "203.0.113.10/32"
instance_count  = 2
subscription_id = null
```

With those defaults, `local.prefix` becomes `clz-dev-clz230`. The load balancer public IP becomes `clz-dev-clz230-lb-pip`, the load balancer becomes `clz-dev-clz230-web-lb`, and the first backend NIC becomes `clz-dev-clz230-web-1-nic`.

The HTTP path remains the normal web path:

```
resource "azurerm_lb_rule" "http" {
  name                           = "http-rule"
  loadbalancer_id                = azurerm_lb.web.id
  protocol                       = "Tcp"
  frontend_port                  = 80
  backend_port                   = 80
  frontend_ip_configuration_name = "public"
  backend_address_pool_ids       = [azurerm_lb_backend_address_pool.web.id]
  probe_id                       = azurerm_lb_probe.http.id
}
```

The NAT rule is separate from the load balancing rule:

```
resource "azurerm_lb_nat_rule" "rdp_first" {
  name                           = "rdp-first-backend"
  resource_group_name            = azurerm_resource_group.lab.name
  loadbalancer_id                = azurerm_lb.web.id
  protocol                       = "Tcp"
  frontend_port                  = 50001
  backend_port                   = 3389
  frontend_ip_configuration_name = "public"
}
```

The frontend port is `50001`; the backend port is `3389`. That distinction matters. Users do not connect to backend port `3389` on the load balancer. They connect to the load balancer public IP on frontend port `50001`, and Azure forwards that session to port `3389` on the selected backend.

The NAT association selects the first backend NIC:

```
resource "azurerm_network_interface_nat_rule_association" "rdp_first" {
  network_interface_id  = azurerm_network_interface.web[0].id
  ip_configuration_name = "internal"
  nat_rule_id           = azurerm_lb_nat_rule.rdp_first.id
}
```

This is intentionally narrow. Only `azurerm_network_interface.web[0]` receives the NAT rule. The second backend remains reachable through HTTP load balancing but does not receive this RDP NAT association.

The outputs provide the access coordinates without exposing the password:

```
output "load_balancer_public_ip" {
  description = "Load balancer public IP."
  value       = azurerm_public_ip.lb.ip_address
}
output "rdp_nat_port" {
  description = "RDP NAT frontend port."
  value       = 50001
}
```

The generated administrator password remains sensitive:

```
output "windows_admin_password" {
  description = "Generated Windows admin password."
  value       = random_password.windows_admin.result
  sensitive   = true
}
```

Run the lab from `CLZ-230-load-balancer-nat-rules`:

```
terraform init
terraform fmt -check
terraform validate
terraform plan -out tfplan
terraform apply tfplan
terraform output
terraform destroy
```

Before apply, confirm the public frontend has two separate purposes: HTTP load balancing on port `80` and a single RDP NAT rule on frontend port `50001`. Confirm that the NAT association targets the first NIC and the `internal` IP configuration. Also confirm that `allow-admin-rdp` still uses `var.admin_cidr`; a NAT rule forwards traffic, but the NSG still controls whether that traffic is allowed.

After apply, the lab RDP target is the load balancer public IP with port `50001`. In PowerShell, the access values can be read separately:

```
$publicIp = terraform output -raw load_balancer_public_ip
$rdpPort = terraform output -raw rdp_nat_port
"$publicIp`:$rdpPort"
```

Cleanup uses `terraform destroy` from the lab folder. Do not commit copied `terraform.tfvars`, saved plans, state files, or retrieved password values. This chapter is about a controlled lab NAT pattern; it does not replace private administration through Bastion-style access.

By the end of this chapter, the load balancer supports both shared HTTP delivery and a deliberate per-instance RDP route. The same repeated-VM shape also shows why Terraform repetition needs careful resource addressing instead of copied blocks.

Chapter 15: Count Windows VMs

This chapter is based on the `CLZ-240-count-windows-vms` lab. The lab creates a small set of identical Windows VMs from a count value.

Manual repetition looks simple until the second or third VM differs from the first. A name is typed slightly differently, a public IP setting is missed, a tag is changed, or a NIC points to the wrong subnet. Terraform `count` gives the lab a simple repetition model: define one resource pattern, then create a numbered set of identical instances.

CLZ-240 uses `var.instance_count` to create repeated public IPs, NICs, and Windows VMs. The default value is `2`, so the lab creates two of each repeated resource. The resources are addressed by numeric index: `[0]`, `[1]`, and so on. That makes `count` appropriate when the instances are intentionally alike and the number is the main input.

The safe example values identify this lab:

```
environment     = "dev"
location        = "eastus2"
lab_id          = "CLZ-240"
name_prefix     = "clz"
admin_username  = "clzadmin"
admin_cidr      = "203.0.113.10/32"
instance_count  = 2
subscription_id = null
```

With those defaults, `local.prefix` becomes `clz-dev-clz240`. The first VM becomes `clz-dev-clz240-count-1-winvm`, and the second becomes `clz-dev-clz240-count-2-winvm`. Their computer names become `clz2401` and `clz2402`.

The counted public IP block creates one public IP per instance:

```
resource "azurerm_public_ip" "web" {
  count               = var.instance_count
  name                = "${local.prefix}-count-${count.index + 1}-pip"
  location            = azurerm_resource_group.lab.location
  resource_group_name = azurerm_resource_group.lab.name
  allocation_method   = "Static"
  sku                 = "Standard"
  tags                = local.tags
}
```

The NIC block repeats the same count and references the matching public IP with `count.index`:

```
resource "azurerm_network_interface" "web" {
  count               = var.instance_count
  name                = "${local.prefix}-count-${count.index + 1}-nic"
  location            = azurerm_resource_group.lab.location
  resource_group_name = azurerm_resource_group.lab.name
  ip_configuration {
    name                          = "internal"
    subnet_id                     = azurerm_subnet.web.id
    private_ip_address_allocation = "Dynamic"
    public_ip_address_id          = azurerm_public_ip.web[count.index].id
  }
  tags = local.tags
}
```

The VM block uses the same index to attach the matching NIC:

```
resource "azurerm_windows_virtual_machine" "web" {
```

```
  count               = var.instance_count
  name                = "${local.prefix}-count-${count.index + 1}-winvm"
  computer_name       = "clz${local.lab_number}${count.index + 1}"
  location            = azurerm_resource_group.lab.location
  resource_group_name = azurerm_resource_group.lab.name
  size                = "Standard_B2s"
  admin_username      = var.admin_username
  admin_password      = random_password.windows_admin.result
  network_interface_ids = [
    azurerm_network_interface.web[count.index].id
  ]
}
```

The repeated VMs still use the standard Windows image, `Standard_B2s`, generated administrator password, and shared tags. The important lesson is not a new VM shape; it is how Terraform expands one shape into a numbered set.

The output uses a splat expression to return the public IPs as a list:

```
output "windows_vm_public_ips" {
  description = "Public IPs created with count."
  value       = azurerm_public_ip.web[*].ip_address
}
```

Run the lab from `CLZ-240-count-windows-vms`:

```
terraform init
terraform fmt -check
terraform validate
terraform plan -out tfplan
terraform apply tfplan
terraform output
terraform destroy
```

Before apply, review the number of planned instances. With `instance_count = 2`, the plan should include two public IPs, two NICs, and two Windows VMs. If `instance_count` is increased, the plan should add new indexed resources. If it is reduced, the plan should destroy the highest indexed resources. That behavior is clear when instances are interchangeable, but it can be risky when each VM has a distinct role.

Cleanup uses `terraform destroy` from the lab folder. Do not commit copied `terraform.tfvars`, saved plans, state files, or retrieved passwords. By the end of this chapter, the reader can use `count` for simple repeated Windows VM patterns. For identical sets, numeric indexes are compact and readable. When machines need stable names or roles, a keyed pattern is safer.

Chapter 16: For Each Windows VMs

This chapter is based on the `CLZ-250-for-each-windows-vms` lab. The lab creates a map-driven set of named Windows VMs.

`count` is useful for a numbered set of identical Windows VMs. That works when the instances are interchangeable. It becomes weaker when the instances have names, roles, or long-lived identities. `for_each` uses stable keys instead of numeric indexes, so Terraform can track resources by meaning rather than position.

CLZ-250 uses the `vm_names` map. The default keys are `web` and `ops`. The current Terraform uses those keys in resource names, public IP names, NIC names, VM names, and output keys.

The map input is:

```
variable "vm_names" {
  description = "Named Windows VM set for for_each examples."
  type        = map(string)
  default = {
    web = "web"
    ops = "ops"
  }
}
```

The public IP block creates one IP per key:

```
resource "azurerm_public_ip" "web" {
  for_each            = var.vm_names
  name                = "${local.prefix}-${each.key}-pip"
  location            = azurerm_resource_group.lab.location
  resource_group_name = azurerm_resource_group.lab.name
  allocation_method   = "Static"
  sku                 = "Standard"
  tags                = local.tags
}
```

The NIC block uses the same key to attach the matching public IP:

```
resource "azurerm_network_interface" "web" {
  for_each            = var.vm_names
  name                = "${local.prefix}-${each.key}-nic"
  location            = azurerm_resource_group.lab.location
  resource_group_name = azurerm_resource_group.lab.name
  ip_configuration {
    name                          = "internal"
    subnet_id                     = azurerm_subnet.web.id
    private_ip_address_allocation = "Dynamic"
    public_ip_address_id          = azurerm_public_ip.web[each.key].id
  }
  tags = local.tags
}
```

The VM block is also keyed:

```
resource "azurerm_windows_virtual_machine" "web" {
  for_each            = var.vm_names
  name                = "${local.prefix}-${each.key}-winvm"
  computer_name       = "clz${local.lab_number}${substr(each.key, 0, 3)}"
  location            = azurerm_resource_group.lab.location
  resource_group_name = azurerm_resource_group.lab.name
  size                = "Standard_B2s"
  admin_username      = var.admin_username
  admin_password      = random_password.windows_admin.result
```

```
  network_interface_ids = [
    azurerm_network_interface.web[each.key].id
  ]
}
```

With default values, Terraform plans `azurerm_windows_virtual_machine.web["web"]` and `azurerm_windows_virtual_machine.web["ops"]`. The Azure VM names become `clz-dev-clz250-web-winvm` and `clz-dev-clz250-ops-winvm`. Their computer names become `clz250web` and `clz250ops`.

The output is a map, not a list:

```
output "windows_vm_public_ips" {
  description = "Public IPs created with for_each."
  value       = { for name, pip in azurerm_public_ip.web : name => pip.ip_address }
}
```

That output shape matches the resource model. A reader does not have to remember whether the first IP belongs to web or ops; the key carries that meaning.

Run the lab from `CLZ-250-for-each-windows-vms`:

```
terraform init
terraform fmt -check
terraform validate
terraform plan -out tfplan
terraform apply tfplan
terraform output
terraform destroy
```

Before apply, confirm the plan shows keyed resources. Adding a key to `vm_names` should add a new named set of resources. Removing a key should destroy that specific named set. Renaming a key is a replacement operation, so it should be reviewed carefully.

Cleanup uses `terraform destroy` from the lab folder. Do not commit copied `terraform.tfvars`, saved plans, state files, or retrieved passwords. By the end of this chapter, the reader has both repetition tools: `count` for simple identical sets and `for_each` for named, stable resources. For larger web tiers, repeated compute is often better expressed as a managed scale set than as many individually addressed VMs.

Chapter 17: Windows VMSS Manual Scaling

This chapter is based on the `CLZ-260-windows-vmss-manual-scaling` lab. The lab creates a Windows VMSS with IIS behind a Standard Load Balancer.

The last two chapters showed how Terraform can create multiple individual VMs with `count` and `for_each`. A VM Scale Set is different. Instead of managing each VM as its own top-level resource, the scale set defines a group of similar instances and controls capacity through the scale set configuration.

CLZ-260 builds a Windows VM Scale Set behind a Standard Load Balancer. The load balancer still provides the public HTTP frontend, but the backend pool is attached through the VMSS network profile. The scale set installs IIS through a VMSS extension, so every instance receives the same web server bootstrap.

The safe example values identify this lab:

```
environment     = "dev"
location        = "eastus2"
lab_id          = "CLZ-260"
name_prefix     = "clz"
admin_username  = "clzadmin"
admin_cidr      = "203.0.113.10/32"
instance_count  = 2
subscription_id = null
```

With those defaults, `local.prefix` becomes `clz-dev-clz260`. The public IP becomes `clz-dev-clz260-vmss-lb-pip`, the load balancer becomes `clz-dev-clz260-vmss-lb`, the backend pool is `vmss-backend-pool`, and the scale set becomes `clz-dev-clz260-web-vmss`.

The load balancer frontend is familiar:

```
resource "azurerm_lb" "web" {
  name                = "${local.prefix}-vmss-lb"
  location            = azurerm_resource_group.lab.location
  resource_group_name = azurerm_resource_group.lab.name
  sku                 = "Standard"
  tags                = local.tags
  frontend_ip_configuration {
    name                 = "public"
    public_ip_address_id = azurerm_public_ip.lb.id
  }
}
```

The VMSS is the main change:

```
resource "azurerm_windows_virtual_machine_scale_set" "web" {
  name                 = "${local.prefix}-web-vmss"
  computer_name_prefix = "clzwin"
  location             = azurerm_resource_group.lab.location
  resource_group_name  = azurerm_resource_group.lab.name
  sku                  = "Standard_B2s"
  instances            = var.instance_count
  admin_username       = var.admin_username
  admin_password       = random_password.windows_admin.result
  upgrade_mode         = "Manual"
  tags                 = local.tags
}
```

The important manual scaling input is `instances = var.instance_count`. With the default value, Terraform plans two VMSS instances. Changing `instance_count` is the manual scale action for this lab. The plan should be reviewed before apply so capacity changes are intentional.

The scale set network profile attaches instances to the web subnet and backend pool:

```
network_interface {
  name    = "web-nic"
  primary = true
  ip_configuration {
    name                                   = "internal"
    primary                                = true
    subnet_id                              = azurerm_subnet.web.id
    load_balancer_backend_address_pool_ids = [azurerm_lb_backend_address_pool.web.id]
  }
}
```

This replaces the per-VM backend pool association pattern used with individually managed backend VMs. Individual VM NIC resources are no longer the main unit. The VMSS network configuration describes how instances join the backend pool.

IIS is installed through a VMSS extension:

```
resource "azurerm_virtual_machine_scale_set_extension" "iis" {
  name                         = "install-iis"
  virtual_machine_scale_set_id = azurerm_windows_virtual_machine_scale_set.web.id
  publisher                    = "Microsoft.Compute"
  type                         = "CustomScriptExtension"
  type_handler_version         = "1.10"
  auto_upgrade_minor_version   = true
  settings = jsonencode({
    commandToExecute = "powershell -ExecutionPolicy Unrestricted -Command \"Install-
WindowsFeature -Name Web-Server -IncludeManagementTools; Set-Content -Path
C:\\inetpub\\wwwroot\\index.html -Value 'Azure From Zero To Hero VMSS ${local.prefix}'\""
  })
}
```

The validation output is the VMSS load balancer URL:

```
output "vmss_load_balancer_url" {
  description = "VMSS load balancer validation URL."
  value       = "http://${azurerm_public_ip.lb.ip_address}"
}
```

Run the lab from `CLZ-260-windows-vmss-manual-scaling`:

```
terraform init
terraform fmt -check
terraform validate
terraform plan -out tfplan
terraform apply tfplan
terraform output
terraform destroy
```

Before apply, confirm the VMSS name, `Standard_B2s` SKU, `instances = 2`, `upgrade_mode = "Manual"`, backend pool attachment, IIS extension, and sensitive password output. After apply, request the output URL and expect the VMSS IIS page. To scale manually, change `instance_count`, run a new plan, and review the capacity change before apply.

Cleanup uses `terraform destroy` from the lab folder. Do not commit copied `terraform.tfvars`, saved plans, state files, or retrieved passwords. By the end of this chapter, the reader has moved

from separate repeated VMs to a managed Windows scale set. Capacity still changes through Terraform input and review, which makes the scaling action deliberate but manual.

Chapter 18: Windows VMSS Autoscaling

This chapter is based on the `CLZ-270-windows-vmss-autoscaling` lab. The lab creates a Windows VMSS with CPU-based scale-out and scale-in rules.

This lab treats the Windows VMSS as an elastic web tier instead of a fixed set of servers. A scale set gives the environment a consistent group of Windows instances behind a load balancer, but fixed capacity still leaves an operator guessing: too few instances create pressure during demand spikes, while too many instances waste budget when traffic is quiet. Autoscaling turns that decision into a monitored control loop. Azure Monitor evaluates a metric over defined time windows, compares the result with scale-out and scale-in thresholds, and changes capacity only inside the minimum, default, and maximum limits declared in Terraform.

CLZ-270 uses a Windows VMSS behind a Standard Load Balancer as the baseline application pattern. The lab still builds the VNet, subnets, NSG rules, Standard public IP, Standard Load Balancer, backend pool, TCP probe, HTTP rule, Windows VMSS, generated password, and IIS extension. The important addition is `azurerm_monitor_autoscale_setting.web`, which binds the autoscale policy to the scale set.

The safe example values identify this lab:

```
environment     = "dev"
location        = "eastus2"
lab_id          = "CLZ-270"
name_prefix     = "clz"
admin_username  = "clzadmin"
admin_cidr      = "203.0.113.10/32"
instance_count  = 2
subscription_id = null
```

With those defaults, `local.prefix` becomes `clz-dev-clz270`. The public IP becomes `clz-dev-clz270-vmss-lb-pip`, the load balancer becomes `clz-dev-clz270-vmss-lb`, the backend pool is `vmss-backend-pool`, the scale set becomes `clz-dev-clz270-web-vmss`, and the autoscale setting becomes `clz-dev-clz270-vmss-autoscale`.

The VMSS remains the compute target:

```
resource "azurerm_windows_virtual_machine_scale_set" "web" {
  name                 = "${local.prefix}-web-vmss"
  computer_name_prefix = "clzwin"
  location             = azurerm_resource_group.lab.location
  resource_group_name  = azurerm_resource_group.lab.name
  sku                  = "Standard_B2s"
  instances            = var.instance_count
  admin_username       = var.admin_username
  admin_password       = random_password.windows_admin.result
  upgrade_mode         = "Manual"
  tags                 = local.tags
}
```

The autoscale setting attaches to that VMSS through `target_resource_id`:

```
resource "azurerm_monitor_autoscale_setting" "web" {
  name                = "${local.prefix}-vmss-autoscale"
  location            = azurerm_resource_group.lab.location
```

```
  resource_group_name = azurerm_resource_group.lab.name
  target_resource_id  = azurerm_windows_virtual_machine_scale_set.web.id
  tags                = local.tags
}
```

The profile defines the capacity boundary:

```
capacity {
  default = tostring(var.instance_count)
  minimum = "2"
  maximum = "4"
}
```

The default capacity follows `var.instance_count`, which is `2` in the example file. Autoscale can reduce only to `2` and increase only to `4`. Those limits matter because autoscale is not just about growth; it is also about cost and blast-radius control.

The scale-out rule watches average CPU:

```
metric_trigger {
  metric_name        = "Percentage CPU"
  metric_resource_id = azurerm_windows_virtual_machine_scale_set.web.id
  time_grain         = "PT1M"
  statistic          = "Average"
  time_window        = "PT5M"
  time_aggregation   = "Average"
  operator           = "GreaterThan"
  threshold          = 70
}
scale_action {
  direction = "Increase"
  type      = "ChangeCount"
  value     = "1"
  cooldown  = "PT5M"
}
```

When average `Percentage CPU` is greater than `70` over the evaluation window, Azure increases the instance count by `1`. The cooldown is `PT5M`, so Azure waits five minutes before another scaling action from that rule.

The scale-in rule uses the same metric with a lower threshold:

```
metric_trigger {
  metric_name        = "Percentage CPU"
  metric_resource_id = azurerm_windows_virtual_machine_scale_set.web.id
  time_grain         = "PT1M"
  statistic          = "Average"
  time_window        = "PT5M"
  time_aggregation   = "Average"
  operator           = "LessThan"
  threshold          = 25
}
scale_action {
  direction = "Decrease"
  type      = "ChangeCount"
  value     = "1"
  cooldown  = "PT5M"
}
```

When average CPU is less than `25`, Azure decreases capacity by `1`, but never below the minimum of`2`. The two rules create a simple operating envelope: add capacity under sustained CPU pressure and remove capacity when the workload cools down.

The outputs expose both the autoscale setting and the web validation URL:

```
output "autoscale_setting_name" {
```

```
  description = "Autoscale setting name."
  value       = azurerm_monitor_autoscale_setting.web.name
}
output "vmss_load_balancer_url" {
  description = "VMSS load balancer validation URL."
  value       = "http://${azurerm_public_ip.lb.ip_address}"
}
```

The generated administrator password remains sensitive:

```
output "windows_admin_password" {
  description = "Generated Windows admin password."
  value       = random_password.windows_admin.result
  sensitive   = true
}
```

Run the lab from `CLZ-270-windows-vmss-autoscaling`:

```
terraform init
terraform fmt -check
terraform validate
terraform plan -out tfplan
terraform apply tfplan
terraform output
terraform destroy
```

Before apply, review the autoscale contract. The setting should target the VMSS, use profile name `default`, set capacity default to `instance_count`, set minimum `2`, set maximum `4`, scale out above CPU `70`, scale in below CPU `25`, change capacity by `1`, and use a five-minute cooldown. Also confirm that the VMSS still joins the load balancer backend pool and that the IIS extension remains present.

After apply, validate the VMSS load balancer URL and inspect the autoscale setting in Azure Monitor or the VMSS scaling blade. The goal is not to force a scale event during every lab run; the goal is to confirm that the scale rules are present, targeted to the correct VMSS, and bounded correctly.

Cleanup uses `terraform destroy` from the lab folder. Do not commit copied `terraform.tfvars`, saved plans, state files, or retrieved password values. By the end of this chapter, the environment uses metric-driven capacity control instead of manual VMSS sizing. Terraform defines both the Windows web tier and the guardrails Azure Monitor must follow when adjusting capacity.

Chapter 19: NAT Gateway Outbound

This chapter is based on the `CLZ-280-nat-gateway-outbound` lab. The lab creates a VNet, four subnets, a Standard public IP, a NAT Gateway, and a subnet association for controlled outbound access.

Outbound access is easy to overlook because it often works before anyone designs it. A private workload may need to reach Windows Update, package sources, certificate endpoints, monitoring services, or an external API. Those requests leave the virtual network, and the source address matters for firewall rules, partner allowlists, troubleshooting, and audit evidence. NAT Gateway turns outbound access into an explicit subnet-level design instead of an implicit side effect.

CLZ-280 keeps the network small enough to inspect. The lab creates one resource group, one VNet, and four subnets: `web-snet`, `app-snet`, `data-snet`, and `mgmt-snet`. The outbound pattern is attached only to the web subnet. That scope matters because NAT Gateway is associated with subnets, not with the whole VNet by default.

The safe example values identify the lab:

```
environment     = "dev"
location        = "eastus2"
lab_id          = "CLZ-280"
name_prefix     = "clz"
admin_username  = "clzadmin"
admin_cidr      = "203.0.113.10/32"
instance_count  = 2
subscription_id = null
```

With those defaults, `local.prefix` becomes `clz-dev-clz280`. The resource group becomes `clz-dev-clz280-rg`, the VNet becomes `clz-dev-clz280-vnet`, the NAT public IP becomes `clz-dev-clz280-nat-pip`, and the NAT Gateway becomes `clz-dev-clz280-natgw`.

The VNet keeps the established address plan:

```
resource "azurerm_virtual_network" "main" {
  name                = "${local.prefix}-vnet"
  address_space       = ["10.40.0.0/16"]
  location            = azurerm_resource_group.lab.location
  resource_group_name = azurerm_resource_group.lab.name
  tags                = local.tags
}
```

The web subnet is the egress target in this lab:

```
resource "azurerm_subnet" "web" {
  name                 = "web-snet"
  resource_group_name  = azurerm_resource_group.lab.name
  virtual_network_name = azurerm_virtual_network.main.name
  address_prefixes     = ["10.40.1.0/24"]
}
```

The NAT Gateway needs a Standard public IP. Static allocation is important because the public IP becomes the stable outbound identity:

```
resource "azurerm_public_ip" "nat" {
  name                = "${local.prefix}-nat-pip"
```

```
  location            = azurerm_resource_group.lab.location
  resource_group_name = azurerm_resource_group.lab.name
  allocation_method   = "Static"
  sku                 = "Standard"
  tags                = local.tags
}
```

The NAT Gateway itself is a regional Standard resource:

```
resource "azurerm_nat_gateway" "main" {
  name                = "${local.prefix}-natgw"
  location            = azurerm_resource_group.lab.location
  resource_group_name = azurerm_resource_group.lab.name
  sku_name            = "Standard"
  tags                = local.tags
}
```

The public IP association gives the gateway its outbound address:

```
resource "azurerm_nat_gateway_public_ip_association" "main" {
  nat_gateway_id       = azurerm_nat_gateway.main.id
  public_ip_address_id = azurerm_public_ip.nat.id
}
```

The subnet association applies the egress path to `web-snet`:

```
resource "azurerm_subnet_nat_gateway_association" "web" {
  subnet_id      = azurerm_subnet.web.id
  nat_gateway_id = azurerm_nat_gateway.main.id
}
```

This association is the operating boundary. Workloads placed in the web subnet use the NAT Gateway for outbound internet traffic. The app, data, and management subnets are still created, but they are not attached to the gateway in this lab. That makes the design easy to review: one subnet receives the controlled egress path, and the other subnets remain separate.

The outputs expose the standard lab identity and the NAT Gateway public IP:

```
output "nat_gateway_public_ip" {
  description = "NAT Gateway outbound public IP."
  value       = azurerm_public_ip.nat.ip_address
}
```

Run the lab from `CLZ-280-nat-gateway-outbound`:

```
terraform init
terraform fmt -check
terraform validate
terraform plan -out tfplan
terraform apply tfplan
terraform output
terraform destroy
```

Before apply, review the plan for the resource group, VNet, four subnets, Standard static public IP, Standard NAT Gateway, NAT Gateway public IP association, and web subnet association. The public IP should be named from the lab prefix, the gateway should use `sku_name = "Standard"`, and the only subnet association should target `azurerm_subnet.web.id`.

After apply, `terraform output nat_gateway_public_ip` should return the outbound public address. In a larger environment, that address can be used in allowlists and operational notes. In this lab, it proves that Terraform created a named egress identity instead of leaving outbound behavior undocumented.

Cleanup uses `terraform destroy` from the lab folder. Do not commit copied `terraform.tfvars`, saved plans, state files, or local output captures. By the end of this chapter, outbound access is a Terraform-managed network decision: a subnet has a deliberate NAT Gateway path, a stable public IP, and a clear validation output.

Chapter 20: Private DNS

This chapter is based on the `CLZ-290-private-dns` lab. The lab creates an internal DNS zone, links it to the lab VNet, and adds a private A record.

Private networks still need names. IP addresses are useful for routing, but they are weak as long-term operating handles. A private record can tell an operator what a target is meant to represent while still resolving to an internal address that never belongs in public DNS. Azure Private DNS gives the VNet a private namespace, and Terraform makes the zone, link, and records reviewable.

CLZ-290 builds a small internal DNS pattern. The lab creates a resource group, a VNet with four subnets, a private DNS zone named `clz.internal`, one virtual network link, and one A record named `web`. The record points to `10.40.1.10`, which keeps the example concrete without depending on a live VM.

The safe example values identify the lab:

```
environment     = "dev"
location        = "eastus2"
lab_id          = "CLZ-290"
name_prefix     = "clz"
admin_username  = "clzadmin"
admin_cidr      = "203.0.113.10/32"
instance_count  = 2
subscription_id = null
```

With those defaults, `local.prefix` becomes `clz-dev-clz290`. The resource group becomes `clz-dev-clz290-rg`, the VNet becomes `clz-dev-clz290-vnet`, and the private DNS VNet link becomes `clz-dev-clz290-vnet-link`.

The VNet gives the private DNS zone a network boundary:

```
resource "azurerm_virtual_network" "main" {
  name                = "${local.prefix}-vnet"
  address_space       = ["10.40.0.0/16"]
  location            = azurerm_resource_group.lab.location
  resource_group_name = azurerm_resource_group.lab.name
  tags                = local.tags
}
```

The private zone is declared directly:

```
resource "azurerm_private_dns_zone" "internal" {
  name                = "clz.internal"
  resource_group_name = azurerm_resource_group.lab.name
  tags                = local.tags
}
```

The name `clz.internal` is intentionally private. It is not delegated from a registrar, and it is not meant to publish records to the public internet. Azure resolves it for resources that can use the linked private DNS zone.

The VNet link connects the zone to the lab network:

```
resource "azurerm_private_dns_zone_virtual_network_link" "internal" {
  name                  = "${local.prefix}-vnet-link"
  resource_group_name   = azurerm_resource_group.lab.name
```

```
  private_dns_zone_name = azurerm_private_dns_zone.internal.name
  virtual_network_id    = azurerm_virtual_network.main.id
  registration_enabled  = false
  tags                  = local.tags
}
```

`registration_enabled = false` is a deliberate choice. Azure will not automatically create records for VMs in the linked network. That keeps this lab focused on explicitly managed records: if a name exists, it exists because Terraform declared it.

The A record creates the internal web name:

```
resource "azurerm_private_dns_a_record" "web" {
  name                = "web"
  zone_name           = azurerm_private_dns_zone.internal.name
  resource_group_name = azurerm_resource_group.lab.name
  ttl                 = 300
  records             = ["10.40.1.10"]
  tags                = local.tags
}
```

The full private name is `web.clz.internal`. The record value is `10.40.1.10`, and the TTL is `300`. This is not automatic service discovery; it is a controlled DNS record that can be reviewed, changed, and removed through Terraform.

The output exposes the zone name:

```
output "private_dns_zone_name" {
  description = "Private DNS zone name."
  value       = azurerm_private_dns_zone.internal.name
}
```

Run the lab from `CLZ-290-private-dns`:

```
terraform init
terraform fmt -check
terraform validate
terraform plan -out tfplan
terraform apply tfplan
terraform output
terraform destroy
```

Before apply, review the plan for the resource group, VNet, four subnets, `azurerm_private_dns_zone.internal`, `azurerm_private_dns_zone_virtual_network_link.internal`, and `azurerm_private_dns_a_record.web`. Confirm that the zone is `clz.internal`, the link targets the lab VNet, registration is disabled, and the `web` record maps to `10.40.1.10`.

After apply, `terraform output private_dns_zone_name` should return `clz.internal`. A runtime resolution test should be performed from a host inside the linked VNet because Private DNS is scoped to the network link. From outside that boundary, the private zone is not a public internet DNS answer.

Cleanup uses `terraform destroy` from the lab folder. Do not commit copied `terraform.tfvars`, saved plans, state files, or local output captures. By the end of this chapter, internal name resolution is part of the Terraform-managed network design: the zone, VNet link, and private record are all visible before deployment.

Chapter 21: Public DNS Optional

This chapter is based on the `CLZ-300-public-dns-optional` lab. The lab models an optional public DNS zone controlled by a boolean variable.

Public DNS has a different risk profile from private DNS. A private zone is scoped to an Azure network link. A public DNS zone represents an internet-facing namespace and only makes sense when the operator owns the domain and is ready to delegate it. For that reason, CLZ-300 is intentionally safe by default: it teaches the Terraform pattern without creating a public DNS zone unless the reader explicitly enables it.

The safe example values identify the lab:

```
environment     = "dev"
location        = "eastus2"
lab_id          = "CLZ-300"
name_prefix     = "clz"
admin_username  = "clzadmin"
admin_cidr      = "203.0.113.10/32"
instance_count  = 2
subscription_id = null
```

The DNS-specific example values show the safety posture:

```
create_public_dns_zone             = false
dns_zone_name                      = "example.com"
primary_endpoint_fqdn              = "primary.example.com"
secondary_endpoint_fqdn            = "secondary.example.com"
```

`create_public_dns_zone` defaults to `false`, and `example.com` is only a placeholder. The chapter does not ask the reader to create a zone for a domain they do not own.

The control variable states that requirement directly:

```
variable "create_public_dns_zone" {
  description = "Set true only when you own the DNS zone name."
  type        = bool
  default     = false
}
```

The zone name variable supplies the public DNS name only when the optional resource is enabled:

```
variable "dns_zone_name" {
  description = "Public DNS zone name used when create_public_dns_zone is true."
  type        = string
  default     = "example.com"
}
```

The public DNS zone uses `count` as the safety switch:

```
resource "azurerm_dns_zone" "public" {
  count               = var.create_public_dns_zone ? 1 : 0
  name                = var.dns_zone_name
  resource_group_name = azurerm_resource_group.lab.name
  tags                = local.tags
}
```

When `create_public_dns_zone` is `false`, Terraform creates zero instances of `azurerm_dns_zone.public`. With the default inputs, the lab still creates the tagged resource group, but it does not publish a DNS zone. When the value is `true`, Terraform creates one DNS zone named by `var.dns_zone_name`.

The output has to use the same condition:

```
output "public_dns_name_servers" {
  description = "Name servers for the optional public DNS zone."
  value       = var.create_public_dns_zone ? azurerm_dns_zone.public[0].name_servers : []
}
```

That guard prevents Terraform from trying to read `azurerm_dns_zone.public[0]` when the resource count is zero. With the default values, `public_dns_name_servers` is an empty list. If a real owned zone is enabled and applied, the output returns the Azure name servers that must be configured at the registrar for delegation.

Run the lab from `CLZ-300-public-dns-optional`:

```
terraform init
terraform fmt -check
terraform validate
terraform plan -out tfplan
terraform apply tfplan
terraform output
terraform destroy
```

Before apply, review the plan against the intended mode. In the default safe mode, the plan should create only the lab resource group, and the output preview should show `public_dns_name_servers = []`. In enabled mode, the plan should include `azurerm_dns_zone.public[0]`, and the zone name should be a domain the reader owns. Domain registration and registrar delegation are outside this Terraform lab, but the Azure name-server output is the value needed for that external step.

After apply in default mode, `terraform output public_dns_name_servers` should return an empty list. After apply in enabled mode, it should return Azure DNS name servers for the public zone. Do not treat the presence of Azure name servers as proof that the domain is live on the internet; public resolution also depends on registrar delegation.

Cleanup uses `terraform destroy` from the lab folder. Do not commit copied `terraform.tfvars`, saved plans, state files, domain ownership details, registrar credentials, or name-server notes. By the end of this chapter, optional public DNS is a controlled Terraform pattern: disabled by default, explicit when enabled, and clear about the boundary between Azure DNS and external domain delegation.

Chapter 22: Remote State Storage

This chapter is based on the `CLZ-310-remote-state-storage` lab. The lab creates an Azure Storage account and a private blob container for shared Terraform state.

Remote state is an operating boundary. Local state is useful while learning because it keeps everything in one folder, but shared Terraform work needs a storage location that is not tied to one workstation. Azure Storage gives Terraform a durable place for state files, and the storage account itself should be created, tagged, reviewed, and cleaned up like any other infrastructure resource.

CLZ-310 creates the backend storage foundation. It does not migrate every lab to remote state automatically. Instead, it builds the Azure objects that other Terraform configurations can reference when they need an Azure Storage backend: a resource group, a globally unique storage account, and a private blob container named `tfstate`.

The safe example values identify the lab:

```
environment     = "dev"
location        = "eastus2"
lab_id          = "CLZ-310"
name_prefix     = "clz"
admin_username  = "clzadmin"
admin_cidr      = "203.0.113.10/32"
instance_count  = 2
subscription_id = null
```

With those defaults, `local.prefix` becomes `clz-dev-clz310`, and the resource group becomes `clz-dev-clz310-rg`. Storage account naming needs extra care because names must be globally unique, lowercase, and no longer than 24 characters. The lab solves that with a random suffix:

```
resource "random_string" "suffix" {
  length  = 6
  upper   = false
  special = false
}
```

The storage account name is built from `st`, the compact prefix, and the generated suffix:

```
resource "azurerm_storage_account" "state" {
  name                                                                                =
substr("st${local.compact_prefix}${random_string.suffix.result}", 0, 24)
  resource_group_name      = azurerm_resource_group.lab.name
  location                 = azurerm_resource_group.lab.location
  account_tier             = "Standard"
  account_replication_type = "LRS"
  min_tls_version          = "TLS1_2"
  tags                     = local.tags
}
```

`Standard` and `LRS` keep the lab economical and easy to inspect. `min_tls_version = "TLS1_2"` sets a clear transport-security floor. The storage account may expose sensitive values such as access keys and connection strings in provider internals, so those values should never be copied into the manuscript, notes, or source control.

The container is the actual place where state blobs can live:

```
resource "azurerm_storage_container" "state" {
  name                  = "tfstate"
  storage_account_id    = azurerm_storage_account.state.id
  container_access_type = "private"
}
```

`container_access_type = "private"` matters. Terraform state can contain resource IDs, generated names, and sometimes sensitive metadata. A state container is not a public artifact.

The outputs expose the safe handoff values:

```
output "state_storage_account_name" {
  description = "Storage account for state."
  value       = azurerm_storage_account.state.name
}
output "state_container_name" {
  description = "Storage container for state."
  value       = azurerm_storage_container.state.name
}
```

These outputs are enough to identify the backend location together with the resource group name. They are not storage keys or secrets.

Run the lab from `CLZ-310-remote-state-storage`:

```
terraform init
terraform fmt -check
terraform validate
terraform plan -out tfplan
terraform apply tfplan
terraform output
terraform destroy
```

Before apply, review the plan for the resource group, `random_string.suffix`, `azurerm_storage_account.state`, and `azurerm_storage_container.state`. Confirm that the storage account uses `Standard` tier, `LRS` replication, `TLS1_2`, standard tags, and a private container named `tfstate`.

After apply, record `resource_group_name`, `state_storage_account_name`, and `state_container_name` only where operationally needed. A configuration that reads or stores remote state also needs a key such as `clz-dev.tfstate`, but the key is a naming convention for the blob, not a secret.

Cleanup uses `terraform destroy` from the lab folder. Do not commit copied `terraform.tfvars`, saved plans, state files, storage account keys, connection strings, or backend credentials. By the end of this chapter, Terraform has a managed Azure Storage location that can support shared state without making state files local workstation artifacts.

Chapter 23: Cross Environment State

This chapter is based on the `CLZ-320-cross-environment-state` lab. The lab reads outputs from a shared state file with `terraform_remote_state`.

Cross-environment state is about controlled information sharing. One Terraform configuration may need values produced by another configuration, such as a storage account name, network ID, DNS zone name, or monitoring workspace ID. Copying those values into notes or variables by hand creates a stale dependency. Reading published outputs from remote state makes the dependency explicit and reviewable.

CLZ-320 is intentionally small. It creates its own tagged resource group, then uses a `terraform_remote_state` data source to read outputs from an Azure Storage backend. The lab does not create the shared backend. It expects the backend location to already exist and the input variables to point to the correct resource group, storage account, container, and state key.

The safe example values identify the lab:

```
environment     = "dev"
location        = "eastus2"
lab_id          = "CLZ-320"
name_prefix     = "clz"
admin_username  = "clzadmin"
admin_cidr      = "203.0.113.10/32"
instance_count  = 2
subscription_id = null
```

The remote-state inputs identify the shared state file:

```
remote_state_resource_group_name   = "clz-dev-clz310-rg"
remote_state_storage_account_name  = "replacewithstateaccount"
remote_state_container_name        = "tfstate"
remote_state_key                   = "clz-dev.tfstate"
```

The storage account value is deliberately a placeholder. A full plan cannot read remote state until `remote_state_storage_account_name` is replaced with the real storage account name from the state-storage lab and the other backend values match the actual location. This protects the reader from thinking remote state is just a local syntax feature. It is a live read from a real backend.

The data source uses the AzureRM backend:

```
data "terraform_remote_state" "shared" {
  backend = "azurerm"
  config = {
    resource_group_name  = var.remote_state_resource_group_name
    storage_account_name = var.remote_state_storage_account_name
    container_name       = var.remote_state_container_name
    key                  = var.remote_state_key
  }
}
```

Each setting has a specific job. `resource_group_name` identifies where the state storage account lives. `storage_account_name` identifies the account. `container_name` identifies the

blob container, which defaults to `tfstate`. `key` identifies the state blob, which defaults to `clz-dev.tfstate`.

The local value collects the remote outputs:

```
locals {
  shared_outputs = try(data.terraform_remote_state.shared.outputs, {})
}
```

The output lists the available output names:

```
output "shared_output_keys" {
  description = "Output keys read from shared state."
  value       = keys(local.shared_outputs)
}
```

Listing keys is safer than dumping every remote output into the chapter. It teaches the shape of the dependency while keeping the design focused on contracts. A remote state consumer should depend on stable, intentional outputs, not on the internal resources of the producing configuration.

Run the lab from `CLZ-320-cross-environment-state`:

```
terraform init
terraform fmt -check
terraform validate
terraform plan -out tfplan
terraform output
terraform destroy
```

Before a full plan, update local backend values in `terraform.tfvars` only if a real shared state location exists. The storage account name should come from the remote state storage output, the container should match `tfstate` unless changed deliberately, and the key should name the intended state blob.

With the default placeholder values, `terraform validate` succeeds because the configuration is structurally valid, but `terraform plan` fails when Terraform tries to read the missing backend. That is expected behavior for this lab state: the dependency has not been satisfied. Once the backend exists and the variables point to it, the plan can read the shared state and `shared_output_keys` can show the available output names.

Cleanup uses `terraform destroy` from the lab folder for resources owned by this configuration. Do not commit copied `terraform.tfvars`, saved plans, state files, storage account keys, connection strings, or backend credentials. By the end of this chapter, shared values are no longer manual copy-and-paste facts; they are outputs read through an explicit remote state boundary.

Chapter 24: Traffic Manager Or Front Door

This chapter is based on the `CLZ-330-traffic-manager-or-front-door` lab. The lab creates an Azure Traffic Manager profile with two external endpoints.

Global routing changes the question from "where is the server?" to "which endpoint should this name resolve to right now?" Traffic Manager answers that question through DNS. It does not sit inline as a reverse proxy. It monitors endpoints, applies a routing method, and returns DNS answers that guide clients toward the preferred target.

CLZ-330 uses Traffic Manager with priority routing. That creates a simple failover pattern: the primary endpoint has priority `1`, and the secondary endpoint has priority `2`. If the primary endpoint is healthy, it is preferred. If the primary endpoint is unhealthy, Traffic Manager can return the secondary endpoint instead.

The safe example values identify the lab:

```
environment     = "dev"
location        = "eastus2"
lab_id          = "CLZ-330"
name_prefix     = "clz"
admin_username  = "clzadmin"
admin_cidr      = "203.0.113.10/32"
instance_count  = 2
subscription_id = null
```

The endpoint FQDN values are placeholders:

```
primary_endpoint_fqdn              = "primary.example.com"
secondary_endpoint_fqdn            = "secondary.example.com"
```

Real deployments should point these variables at endpoints the operator owns and can monitor. The placeholders are useful for learning the Terraform shape, but they are not proof of a working global application.

Traffic Manager DNS names need to be unique. The lab uses a random suffix:

```
resource "random_string" "suffix" {
  length  = 6
  upper   = false
  special = false
}
```

The profile is the central resource:

```
resource "azurerm_traffic_manager_profile" "web" {
  name                   = "${local.prefix}-tm"
  resource_group_name    = azurerm_resource_group.lab.name
  traffic_routing_method = "Priority"
  tags                   = local.tags
  dns_config {
    relative_name  =  substr("tm-${local.compact_prefix}-${random_string.suffix.result}",
0, 63)
    ttl           = 60
  }
  monitor_config {
    protocol = "HTTPS"
    port     = 443
    path     = "/"
  }
}
```

`traffic_routing_method = "Priority"` makes lower numbers more preferred. The DNS `relative_name` starts with `tm-`, includes the compact prefix, and adds the random suffix. The TTL is `60`, which keeps cached answers short-lived enough for a lab. The monitor checks HTTPS on port `443` at `/`, so the endpoint must answer that health check to be considered healthy.

The primary endpoint is an external endpoint:

```
resource "azurerm_traffic_manager_external_endpoint" "primary" {
  name       = "primary"
  profile_id = azurerm_traffic_manager_profile.web.id
  target     = var.primary_endpoint_fqdn
  priority   = 1
}
```

The secondary endpoint uses the same profile and a lower preference:

```
resource "azurerm_traffic_manager_external_endpoint" "secondary" {
  name       = "secondary"
  profile_id = azurerm_traffic_manager_profile.web.id
  target     = var.secondary_endpoint_fqdn
  priority   = 2
}
```

The output exposes the Traffic Manager DNS name:

```
output "traffic_manager_fqdn" {
  description = "Traffic Manager DNS name."
  value       = azurerm_traffic_manager_profile.web.fqdn
}
```

Run the lab from `CLZ-330-traffic-manager-or-front-door`:

```
terraform init
terraform fmt -check
terraform validate
terraform plan -out tfplan
terraform apply tfplan
terraform output
terraform destroy
```

Before apply, review the plan for the resource group, random suffix, Traffic Manager profile, and two external endpoints. Confirm `traffic_routing_method = "Priority"`, DNS TTL `60`, HTTPS monitoring on port `443`, monitor path `/`, primary priority `1`, and secondary priority `2`. Also confirm that the endpoint targets are intentional, not leftover placeholder names.

After apply, `terraform output traffic_manager_fqdn` should return the Traffic Manager DNS name. Testing that name depends on the endpoint targets. If the endpoints are placeholders, the profile can exist but the health model will not prove an application failover path. If the endpoints are real and healthy, DNS resolution and endpoint health can be tested from client tools.

Cleanup uses `terraform destroy` from the lab folder. Do not commit copied `terraform.tfvars`, saved plans, state files, domain credentials, endpoint secrets, or registrar notes. By the end of this chapter, global routing is represented as Terraform-managed DNS behavior: a named Traffic Manager profile, monitored endpoints, and an output FQDN that can be validated without hiding the routing policy in portal clicks.

Chapter 25: Key Vault Secrets

This chapter is based on the `CLZ-340-key-vault-secrets` lab. The lab creates a Key Vault and stores a generated Windows administrator password as a secret.

Secret handling is where automation must become more careful, not less. Terraform can generate a strong password, but the generated value is still sensitive. If it is copied into notes, screenshots, shell history, or committed files, the automation has only moved the risk. Key Vault gives the lab a managed destination for the secret while Terraform keeps validation focused on names and resource relationships.

CLZ-340 creates a resource group, a generated Windows administrator password, a Key Vault, and one Key Vault secret named `windows-admin-password`. The chapter does not print the password. The safe outputs are the Key Vault name and the stored secret name.

The safe example values identify the lab:

```
environment     = "dev"
location        = "eastus2"
lab_id          = "CLZ-340"
name_prefix     = "clz"
admin_username  = "clzadmin"
admin_cidr      = "203.0.113.10/32"
instance_count  = 2
subscription_id = null
```

With those defaults, `local.prefix` becomes `clz-dev-clz340`, and the Key Vault name becomes `kv-clzdevclz340`.

The password resource defines the credential shape:

```
resource "random_password" "windows_admin" {
  length           = 20
  special          = true
  min_upper        = 2
  min_lower        = 2
  min_numeric      = 2
  min_special      = 2
  override_special = "!#$%&*()-_=+[]{}<>:?"
}
```

The value of `random_password.windows_admin.result` is sensitive. It should flow into a secret resource, not into prose or ordinary outputs.

Terraform reads the current Azure client context:

```
data "azurerm_client_config" "current" {}
```

That data source supplies the tenant and object identity used by the vault and access policy. The chapter should not record the tenant ID or object ID from a real plan output.

The Key Vault resource is declared with the standard SKU:

```
resource "azurerm_key_vault" "lab" {
  name                        = substr("kv-${local.compact_prefix}", 0, 24)
  location                    = azurerm_resource_group.lab.location
  resource_group_name         = azurerm_resource_group.lab.name
  tenant_id                   = data.azurerm_client_config.current.tenant_id
  sku_name                    = "standard"
  soft_delete_retention_days = 7
```

```
  purge_protection_enabled   = false
  tags                       = local.tags
}
```

The lab keeps `purge_protection_enabled = false` so the resource remains easy to remove during cleanup. In a production design, purge protection is usually a stronger safety control. The lab setting is about cleanup predictability, not a universal production recommendation.

The access policy grants the current identity secret operations for the lesson:

```
secret_permissions = [
  "Delete",
  "Get",
  "List",
  "Purge",
  "Recover",
  "Set"
]
```

The secret stores the generated password:

```
resource "azurerm_key_vault_secret" "windows_admin_password" {
  name         = "windows-admin-password"
  value        = random_password.windows_admin.result
  key_vault_id = azurerm_key_vault.lab.id
  tags         = local.tags
}
```

The outputs expose only safe handles:

```
output "key_vault_name" {
  description = "Key Vault name."
  value       = azurerm_key_vault.lab.name
}
output "secret_name" {
  description = "Stored secret name."
  value       = azurerm_key_vault_secret.windows_admin_password.name
}
```

Run the lab from `CLZ-340-key-vault-secrets`:

```
terraform init
terraform fmt -check
terraform validate
terraform plan -out tfplan
terraform apply tfplan
terraform output
terraform destroy
```

Before apply, review the plan for the resource group, `random_password.windows_admin`, `azurerm_key_vault.lab`, and `azurerm_key_vault_secret.windows_admin_password`. Confirm the password length and complexity settings, the vault name, `sku_name = "standard"`, soft-delete retention of `7`, and secret name `windows-admin-password`.

After apply, validate `key_vault_name` and `secret_name`. Do not retrieve or paste the generated password as routine evidence. If the secret must be used operationally, retrieve it through an approved secret-access path and avoid copying it into source control or manuscript files.

Cleanup uses `terraform destroy` from the lab folder. Do not commit copied `terraform.tfvars`, saved plans, state files, generated passwords, vault access details, tenant IDs, object IDs, or secret values. By the end of this chapter, generated credentials have a managed destination:

Terraform creates the password, Key Vault stores it, and the manuscript validates the storage pattern without exposing the secret.

Chapter 26: Private Endpoint Storage

This chapter is based on the `CLZ-350-private-endpoint-storage` lab. The lab creates a storage account, a private endpoint for blob access, and a private DNS zone group.

Private endpoints bring Azure platform services into the private network model. A storage account is still a managed Azure service, but clients in the VNet can reach its blob endpoint through a private IP instead of relying on the public service endpoint. That shift is only complete when DNS resolves the service name to the private endpoint. The network path and the name-resolution path have to work together.

CLZ-350 builds that pattern with a storage account, a VNet, four subnets, the blob Private DNS zone, a VNet link, and a private endpoint in the data subnet. The endpoint connects specifically to the storage account `blob` subresource.

The safe example values identify the lab:

```
environment     = "dev"
location        = "eastus2"
lab_id          = "CLZ-350"
name_prefix     = "clz"
admin_username  = "clzadmin"
admin_cidr      = "203.0.113.10/32"
instance_count  = 2
subscription_id = null
```

The VNet uses the familiar `10.40.0.0/16` address space. The data subnet is the private endpoint subnet:

```
resource "azurerm_subnet" "data" {
  name                 = "data-snet"
  resource_group_name  = azurerm_resource_group.lab.name
  virtual_network_name = azurerm_virtual_network.main.name
  address_prefixes     = ["10.40.3.0/24"]
}
```

The storage account uses a generated suffix because storage account names must be globally unique:

```
resource "azurerm_storage_account" "private" {
  name                                                                    =
substr("st${local.compact_prefix}${random_string.suffix.result}", 0, 24)
  resource_group_name      = azurerm_resource_group.lab.name
  location                 = azurerm_resource_group.lab.location
  account_tier             = "Standard"
  account_replication_type = "LRS"
  min_tls_version          = "TLS1_2"
  tags                     = local.tags
}
```

The Private DNS zone uses Azure's blob privatelink zone name:

```
resource "azurerm_private_dns_zone" "blob" {
  name                = "privatelink.blob.core.windows.net"
  resource_group_name = azurerm_resource_group.lab.name
  tags                = local.tags
}
```

The zone link makes that private zone available to the VNet:

```
resource "azurerm_private_dns_zone_virtual_network_link" "blob" {
  name                  = "${local.prefix}-blob-link"
```

```
  resource_group_name   = azurerm_resource_group.lab.name
  private_dns_zone_name = azurerm_private_dns_zone.blob.name
  virtual_network_id    = azurerm_virtual_network.main.id
  tags                  = local.tags
}
```

The private endpoint is placed in `data-snet`:

```
resource "azurerm_private_endpoint" "blob" {
  name                = "${local.prefix}-blob-pe"
  location            = azurerm_resource_group.lab.location
  resource_group_name = azurerm_resource_group.lab.name
  subnet_id           = azurerm_subnet.data.id
  tags                = local.tags
}
```

The service connection targets the storage account and the `blob` subresource:

```
private_service_connection {
  name                           = "${local.prefix}-blob-psc"
  private_connection_resource_id = azurerm_storage_account.private.id
  subresource_names              = ["blob"]
  is_manual_connection           = false
}
```

`is_manual_connection = false` means the connection is approved automatically for this Terraform-managed resource relationship.

The DNS zone group attaches the private endpoint to the blob Private DNS zone:

```
private_dns_zone_group {
  name                 = "default"
  private_dns_zone_ids = [azurerm_private_dns_zone.blob.id]
}
```

Without that DNS integration, the private endpoint could exist while clients still resolve the storage name through the public path. The zone group keeps the network connection and name resolution aligned.

The outputs expose validation handles:

```
output "storage_account_name" {
  description = "Storage account reached by private endpoint."
  value       = azurerm_storage_account.private.name
}
output "private_endpoint_id" {
  description = "Private endpoint ID."
  value       = azurerm_private_endpoint.blob.id
}
```

Run the lab from `CLZ-350-private-endpoint-storage`:

```
terraform init
terraform fmt -check
terraform validate
terraform plan -out tfplan
terraform apply tfplan
terraform output
terraform destroy
```

Before apply, review the plan for the resource group, VNet, four subnets, random suffix, storage account, blob Private DNS zone, VNet link, private endpoint, service connection to `blob`, and DNS zone group. Confirm that the endpoint subnet is `azurerm_subnet.data.id` and that the DNS zone name is `privatelink.blob.core.windows.net`.

After apply, validate `storage_account_name` and `private_endpoint_id`. A true DNS test should run from inside the linked VNet, because the private zone is scoped to that network. The output alone proves resource creation; internal resolution proves the private access path.

Cleanup uses `terraform destroy` from the lab folder. Do not commit copied `terraform.tfvars`, saved plans, state files, storage account keys, connection strings, private endpoint output captures, or DNS troubleshooting notes with sensitive values. By the end of this chapter, storage access is represented as a private network pattern: service connection, endpoint subnet, Private DNS zone, and VNet link are all visible in Terraform.

Chapter 27: Azure Monitor Log Analytics

This chapter is based on the `CLZ-360-azure-monitor-log-analytics` lab. The lab creates a Log Analytics workspace and an Azure Monitor action group foundation.

Operations need a destination for evidence. A deployed resource can be healthy, degraded, noisy, or failing, but without a workspace and alert-routing pattern the signals stay scattered. Log Analytics gives Azure a place to collect and query operational data. An action group gives alerts a named routing target, even when the lab does not attach real receivers.

CLZ-360 keeps the monitoring foundation intentionally small. It creates a tagged resource group, one Log Analytics workspace, and one action group. The lab does not configure real email addresses, webhooks, SMS numbers, or on-call integrations. That keeps personal contact details out of the repo while still teaching the resource shape.

The safe example values identify the lab:

```
environment     = "dev"
location        = "eastus2"
lab_id          = "CLZ-360"
name_prefix     = "clz"
admin_username  = "clzadmin"
admin_cidr      = "203.0.113.10/32"
instance_count  = 2
subscription_id = null
```

With those defaults, `local.prefix` becomes `clz-dev-clz360`. The workspace becomes `clz-dev-clz360-law`, and the action group becomes `clz-dev-clz360-ops-ag`.

The Log Analytics workspace is the data foundation:

```
resource "azurerm_log_analytics_workspace" "main" {
  name                = "${local.prefix}-law"
  location            = azurerm_resource_group.lab.location
  resource_group_name = azurerm_resource_group.lab.name
  sku                 = "PerGB2018"
  retention_in_days   = 30
  tags                = local.tags
}
```

`sku = "PerGB2018"` is the standard consumption-oriented workspace SKU used in many small environments. `retention_in_days = 30` gives the lab a clear retention setting instead of leaving the value implicit. The workspace also has shared keys in provider-managed attributes, so validation should use the workspace ID output, not copied keys.

The action group establishes an alert-routing object:

```
resource "azurerm_monitor_action_group" "ops" {
  name                = "${local.prefix}-ops-ag"
  resource_group_name = azurerm_resource_group.lab.name
  short_name          = "clzops"
  tags                = local.tags
}
```

The short name `clzops` is required by Azure Monitor for compact display and notification contexts. This lab does not add receivers. A real environment can add email, webhook, ITSM,

Azure Function, Logic App, or other receiver types, but those values often contain personal or operational contact details and should be handled intentionally.

The output exposes the workspace ID:

```
output "log_analytics_workspace_id" {
  description = "Log Analytics workspace ID."
  value       = azurerm_log_analytics_workspace.main.id
}
```

Run the lab from `CLZ-360-azure-monitor-log-analytics`:

```
terraform init
terraform fmt -check
terraform validate
terraform plan -out tfplan
terraform apply tfplan
terraform output
terraform destroy
```

Before apply, review the plan for the resource group, `azurerm_log_analytics_workspace.main`, and `azurerm_monitor_action_group.ops`. Confirm the workspace name, SKU `PerGB2018`, retention `30`, action group name, and short name `clzops`. Also confirm that no personal receiver details were added accidentally.

After apply, validate `log_analytics_workspace_id` and inspect the workspace and action group in Azure Monitor. The workspace ID proves the destination exists. Actual log collection requires diagnostic settings, agents, or service integrations that send data into the workspace.

Cleanup uses `terraform destroy` from the lab folder. Do not commit copied `terraform.tfvars`, saved plans, state files, workspace keys, alert contact details, webhook URLs, or operational routing notes. By the end of this chapter, monitoring is no longer an afterthought: the workspace and alert-routing foundation are visible in Terraform before alert rules and diagnostics are layered on top.

Chapter 28: GitHub Actions Terraform

This chapter is based on the `CLZ-370-github-actions-terraform` lab. The lab creates a small Terraform target and adds a GitHub Actions workflow for repeatable Terraform checks.

Terraform should not depend on memory or a lucky terminal session. A pull request needs a consistent review path: format check, provider initialization, validation, and a plan that fails clearly when the code cannot be evaluated. GitHub Actions provides that path directly inside the repository.

CLZ-370 keeps the Azure deployment deliberately small. The Terraform target is a tagged resource group:

```
resource "azurerm_resource_group" "lab" {
  name     = "${local.prefix}-rg"
  location = var.location
  tags     = local.tags
}
```

The lab still uses the normal naming and tagging locals:

```
locals {
  lab_number     = replace(var.lab_id, "CLZ-", "")
  lab_code       = lower(replace(var.lab_id, "CLZ-", "clz"))
  prefix         = lower("${var.name_prefix}-${var.environment}-${local.lab_code}")
  compact_prefix = substr(replace(local.prefix, "-", ""), 0, 18)
  tags = {
    Project     = "Azure From Zero To Hero"
    Environment = var.environment
    ManagedBy   = "Terraform"
    Lab         = var.lab_id
  }
}
```

The safe example values keep the run predictable:

```
environment     = "dev"
location        = "eastus2"
lab_id          = "CLZ-370"
name_prefix     = "clz"
admin_username  = "clzadmin"
admin_cidr      = "203.0.113.10/32"
instance_count  = 2
subscription_id = null
```

With those defaults, `local.prefix` becomes `clz-dev-clz370`, and the resource group becomes `clz-dev-clz370-rg`. The small Azure surface is intentional: the workflow is the real operating lesson.

The lab records the workflow convention in Terraform:

```
locals {
  workflow_folder = ".github/workflows"
}
```

That local value does not create anything in GitHub. It points to the repository convention where workflow files live. The actual automation is the YAML workflow under `.github/workflows`.

The workflow is named `terraform-plan` and can run either by manual dispatch or by pull request:

```
name: terraform-plan
```

```
on:
  workflow_dispatch:
  pull_request:
    paths:
      - "Chriz Labz/**"
```

Path filters are easy to overlook. If the Terraform folder is moved or renamed, the workflow may stop running even though the YAML still looks valid. The `paths` value must match the repository layout that should trigger Terraform review.

The job runs on a Windows-hosted runner with PowerShell and a fixed Terraform working directory:

```
jobs:
  terraform:
    runs-on: windows-latest
    defaults:
      run:
        shell: pwsh
        working-directory: "Chriz Labz/CLZ-370-github-actions-terraform"
```

The working directory decides which Terraform configuration the runner evaluates. If it points to the wrong folder, Terraform will not be checking the lab code. When adapting this workflow, update the pull request path filter and the working directory together.

The Terraform steps are the CI gate:

```
steps:
  - name: Checkout
    uses: actions/checkout@v4
  - name: Setup Terraform
    uses: hashicorp/setup-terraform@v3
  - name: Terraform Init
    run: terraform init -backend=false
  - name: Terraform Format
    run: terraform fmt -check
  - name: Terraform Validate
    run: terraform validate
  - name: Terraform Plan
    run: terraform plan -input=false
```

`terraform init -backend=false` prepares the working directory and provider plugins without requiring a remote state backend. `terraform fmt -check` makes style drift visible. `terraform validate` checks the configuration against Terraform and provider schema rules. `terraform plan -input=false` forces the run to be noninteractive, which is mandatory for a CI runner.

The workflow includes the permissions needed to read the repository and request an OIDC token:

```
permissions:
  id-token: write
  contents: read
```

This is not the same as a complete Azure login. `id-token: write` allows token issuance, but Azure still needs a federated credential and the workflow still needs an explicit login step before a cloud-backed plan can authenticate. The lab teaches the Terraform workflow structure; production CI must add the identity design that matches the target subscription and security model.

Run the lab locally from `CLZ-370-github-actions-terraform`:

```
terraform init
terraform fmt -check
terraform validate
terraform plan -out tfplan
terraform apply tfplan
terraform output
terraform destroy
```

Before apply, confirm that the plan creates only the expected resource group and returns the resource group name and location. In GitHub Actions, troubleshoot failures by checking the trigger path, the working directory, provider initialization, formatting, validation, and Azure authentication in that order.

Do not commit copied `terraform.tfvars`, saved plans, state files, Azure credentials, service principal secrets, federated identity details, or workflow token values. By the end of this chapter, Terraform review is no longer just a local habit. It is a repeatable repository workflow that can be expanded with authentication, plan artifacts, policy checks, and approval gates when the environment is ready.

Chapter 29: Azure SQL Private Access

This chapter is based on the `CLZ-380-azure-sql-private-access` lab. The lab deploys Azure SQL Database with public network access disabled and connects the service to a virtual network through Private Endpoint and private DNS.

Private database access is a routing design, not just a security checkbox. The SQL server still runs as an Azure managed service, but clients should not depend on the public endpoint. They need a private network path and a DNS answer that sends the SQL name toward that private path.

CLZ-380 builds the full shape: a resource group, a virtual network, four subnets, a generated SQL administrator password, an Azure SQL logical server, a Basic database, a private DNS zone, a VNet link, and a private endpoint for the SQL server subresource.

The safe example values define the lab run:

```
environment     = "dev"
location        = "eastus2"
lab_id          = "CLZ-380"
name_prefix     = "clz"
admin_username  = "clzadmin"
admin_cidr      = "203.0.113.10/32"
instance_count  = 2
subscription_id = null
sql_admin_login = "clzsqladmin"
```

With those defaults, `local.prefix` becomes `clz-dev-clz380`. The resource group becomes `clz-dev-clz380-rg`, the virtual network becomes `clz-dev-clz380-vnet`, and the SQL logical server becomes `clz-dev-clz380-sql`.

The virtual network provides the address space for the private access design:

```
resource "azurerm_virtual_network" "main" {
  name                = "${local.prefix}-vnet"
  location            = azurerm_resource_group.lab.location
  resource_group_name = azurerm_resource_group.lab.name
  address_space       = ["10.40.0.0/16"]
  tags                = local.tags
}
```

The lab separates workload roles into subnets. The private endpoint uses the data subnet:

```
resource "azurerm_subnet" "data" {
  name                 = "data-snet"
  resource_group_name  = azurerm_resource_group.lab.name
  virtual_network_name = azurerm_virtual_network.main.name
  address_prefixes     = ["10.40.3.0/24"]
}
```

This placement gives the virtual network a private entry point to Azure SQL. It does not place the database engine inside the subnet. Azure SQL remains managed by Azure; the subnet hosts the private endpoint interface that clients can reach.

The SQL administrator password is generated rather than stored in source:

```
resource "random_password" "windows_admin" {
  length           = 20
  special          = true
```

```
  min_upper        = 2
  min_lower        = 2
  min_numeric      = 2
  min_special      = 2
  override_special = "!#$%&*()-_=+[]{}<>:?"
}
```

The SQL server configuration makes the network decision explicit:

```
resource "azurerm_mssql_server" "main" {
  name                          = "${local.prefix}-sql"
  location                      = azurerm_resource_group.lab.location
  resource_group_name           = azurerm_resource_group.lab.name
  version                       = "12.0"
  administrator_login           = var.sql_admin_login
  administrator_login_password  = random_password.windows_admin.result
  minimum_tls_version           = "1.2"
  public_network_access_enabled = false
  tags                          = local.tags
}
```

`minimum_tls_version = "1.2"` sets the transport baseline. `public_network_access_enabled = false` removes dependency on the public SQL endpoint. If clients need access, they need the private endpoint and DNS path to work.

The database is intentionally small:

```
resource "azurerm_mssql_database" "main" {
  name      = "clzdb"
  server_id = azurerm_mssql_server.main.id
  sku_name  = "Basic"
  tags      = local.tags
}
```

Private DNS supplies the service-specific zone:

```
resource "azurerm_private_dns_zone" "sql" {
  name                = "privatelink.database.windows.net"
  resource_group_name = azurerm_resource_group.lab.name
  tags                = local.tags
}
```

The zone is linked to the virtual network:

```
resource "azurerm_private_dns_zone_virtual_network_link" "sql" {
  name                  = "${local.prefix}-sql-link"
  resource_group_name   = azurerm_resource_group.lab.name
  private_dns_zone_name = azurerm_private_dns_zone.sql.name
  virtual_network_id    = azurerm_virtual_network.main.id
  registration_enabled  = false
  tags                  = local.tags
}
```

The private endpoint binds the private network path to the SQL server:

```
resource "azurerm_private_endpoint" "sql" {
  name                = "${local.prefix}-sql-pe"
  location            = azurerm_resource_group.lab.location
  resource_group_name = azurerm_resource_group.lab.name
  subnet_id           = azurerm_subnet.data.id
  tags                = local.tags
  private_service_connection {
    name                           = "${local.prefix}-sql-psc"
    private_connection_resource_id = azurerm_mssql_server.main.id
    subresource_names              = ["sqlServer"]
    is_manual_connection           = false
  }
  private_dns_zone_group {
    name                 = "default"
    private_dns_zone_ids = [azurerm_private_dns_zone.sql.id]
  }
```

```
}
```

The important relationship is three-part: the SQL server has public access disabled, the private endpoint connects to `sqlServer`, and the DNS zone group ties the endpoint to `privatelink.database.windows.net`. If any one part is missing, the database may exist but private access will not behave as expected.

Run the lab from `CLZ-380-azure-sql-private-access`:

```
terraform init
terraform fmt -check
terraform validate
terraform plan -out tfplan
terraform apply tfplan
terraform output
terraform destroy
```

Before apply, review the plan for the resource group, VNet, four subnets, generated password, SQL server, database, private DNS zone, VNet link, and private endpoint. Confirm the SQL server has TLS 1.2, `public_network_access_enabled = false`, database SKU `Basic`, DNS zone `privatelink.database.windows.net`, DNS zone group `default`, and private endpoint subresource `sqlServer`.

After apply, validate from a network location that can use the linked VNet DNS path. The useful test is not only whether the SQL server exists. The useful test is whether the normal SQL name resolves through private DNS and whether access depends on the private endpoint instead of the public SQL endpoint.

Cleanup uses `terraform destroy` from the lab folder. Do not commit copied `terraform.tfvars`, saved plans, state files, generated passwords, SQL connection strings, or database credentials. By the end of this chapter, Azure SQL has a private access pattern that fits application networks without exposing the database through public network access.

Chapter 30: Final Windows Reference Architecture

This chapter is based on the `CLZ-390-final-windows-reference-architecture` lab. The lab assembles a Windows reference architecture from reusable Terraform modules plus Bastion, Key Vault, and Log Analytics.

A useful reference architecture shows relationships. The network gives the compute layer a place to live. The load balancer gives users a stable HTTP entry point. Bastion provides managed administrative access without opening direct RDP. Key Vault stores the generated administrator password. Log Analytics gives operations a workspace for evidence. CLZ-390 brings those pieces into one Terraform graph.

The safe example values define the deployment:

```
environment     = "dev"
location        = "eastus2"
lab_id          = "CLZ-390"
name_prefix     = "clz"
admin_username  = "clzadmin"
admin_cidr      = "203.0.113.10/32"
instance_count  = 2
subscription_id = null
```

With those defaults, `local.prefix` becomes `clz-dev-clz390`. The resource group becomes `clz-dev-clz390-rg`, the virtual network becomes `clz-dev-clz390-vnet`, the VMSS becomes `clz-dev-clz390-web-vmss`, and the Log Analytics workspace becomes `clz-dev-clz390-law`.

The administrator password is generated inside Terraform:

```
resource "random_password" "windows_admin" {
  length           = 20
  special          = true
  min_upper        = 2
  min_lower        = 2
  min_numeric      = 2
  min_special      = 2
  override_special = "!#$%&*()-_=+[]{}<>:?"
}
```

The value is used by the VMSS module and stored in Key Vault. It is not meant to be copied into the chapter, a README, a ticket, or a committed variables file.

The network foundation comes from the shared `network-core` module:

```
module "network" {
  source              = "../modules/network-core"
  resource_group_name = azurerm_resource_group.lab.name
  location            = azurerm_resource_group.lab.location
  prefix              = local.prefix
  tags                = local.tags
}
```

That module creates the `10.40.0.0/16` virtual network, a web subnet at `10.40.1.0/24`, and a data subnet at `10.40.3.0/24`. Its outputs let the capstone consume subnet IDs without repeating the module's internal resource definitions.

The Windows IIS VMSS module receives the web subnet and generated password:

```
module "windows_iis_vmss" {
```

```
  source              = "../modules/windows-iis-vmss"
  resource_group_name = azurerm_resource_group.lab.name
  location            = azurerm_resource_group.lab.location
  prefix              = local.prefix
  subnet_id           = module.network.web_subnet_id
  admin_username      = var.admin_username
  admin_password      = random_password.windows_admin.result
  instance_count      = var.instance_count
  tags                = local.tags
}
```

Inside the module, Terraform creates a Standard public IP, a Standard load balancer, a backend pool, a TCP health probe on port 80, a load-balancing rule, a Windows Server 2022 VMSS, and a Custom Script Extension that installs IIS. The module output returns the load balancer URL so the deployed web tier has a simple validation target.

Bastion adds managed access. Azure requires the subnet to be named exactly `AzureBastionSubnet`:

```
resource "azurerm_subnet" "bastion" {
  name                 = "AzureBastionSubnet"
  resource_group_name  = azurerm_resource_group.lab.name
  virtual_network_name = module.network.vnet_name
  address_prefixes     = ["10.40.20.0/26"]
}
```

The Bastion host also needs a Standard static public IP:

```
resource "azurerm_public_ip" "bastion" {
  name                = "${local.prefix}-bas-pip"
  location            = azurerm_resource_group.lab.location
  resource_group_name = azurerm_resource_group.lab.name
  allocation_method   = "Static"
  sku                 = "Standard"
  tags                = local.tags
}
```

The host ties the subnet and public IP together:

```
resource "azurerm_bastion_host" "main" {
  name                = "${local.prefix}-bastion"
  location            = azurerm_resource_group.lab.location
  resource_group_name = azurerm_resource_group.lab.name
  tags                = local.tags
  ip_configuration {
    name                 = "configuration"
    subnet_id            = azurerm_subnet.bastion.id
    public_ip_address_id = azurerm_public_ip.bastion.id
  }
}
```

Key Vault is created with a standard SKU and a short retention window for the lab. Terraform reads the current Azure client configuration to build the access policy, then stores the generated administrator password as a secret:

```
resource "azurerm_key_vault_secret" "windows_admin_password" {
  name         = "windows-admin-password"
  value        = random_password.windows_admin.result
  key_vault_id = azurerm_key_vault.lab.id
  tags         = local.tags
}
```

The secret name is safe to document. The secret value is not.

The monitoring foundation is a Log Analytics workspace:

```
resource "azurerm_log_analytics_workspace" "main" {
```

```
  name                = "${local.prefix}-law"
  location            = azurerm_resource_group.lab.location
  resource_group_name = azurerm_resource_group.lab.name
  sku                 = "PerGB2018"
  retention_in_days   = 30
  tags                = local.tags
}
```

The outputs provide the useful handles:

```
output "capstone_load_balancer_url" {
  description = "Capstone VMSS load balancer URL."
  value       = module.windows_iis_vmss.load_balancer_url
}
output "key_vault_name" {
  description = "Capstone Key Vault name."
  value       = azurerm_key_vault.lab.name
}
output "windows_admin_password" {
  description = "Generated Windows admin password."
  value       = random_password.windows_admin.result
  sensitive   = true
}
```

Run the lab from `CLZ-390-final-windows-reference-architecture`:

```
terraform init
terraform fmt -check
terraform validate
terraform plan -out tfplan
terraform apply tfplan
terraform output
terraform destroy
```

Before apply, review the plan for the resource group, generated password, network module resources, VMSS module resources, Bastion subnet, Bastion public IP, Bastion host, Key Vault, Key Vault secret, and Log Analytics workspace. The plan should show the sensitive password as sensitive, not as a copied value.

After apply, validate the load balancer URL, VMSS health, Bastion availability, Key Vault name, secret presence, and Log Analytics workspace. Avoid retrieving the generated password unless it is needed for an operational action, and never paste it into source-controlled notes.

Cleanup uses `terraform destroy` from the lab folder. Do not commit copied `terraform.tfvars`, saved plans, state files, generated passwords, Key Vault secret values, tenant IDs, object IDs, or workspace keys. By the end of this chapter, the book's Windows lab environment has a complete reference architecture: network, compute, administrative access, secret storage, and monitoring all represented as Terraform.

Operating Azure as Code

This outro is based on the repository `README.md`, the final book outline, the manifest, the concept tracker, and the repo validation scripts.

The book ends with an operating model. A deployed VM, a working load balancer, a private endpoint, or a reachable database can prove that a lab succeeded, but the larger skill is repeatability. The useful question is not only "Did the resource appear?" It is "Can the same change be reviewed, rebuilt, validated, and removed without relying on memory?"

That question starts on the workstation. PowerShell, Terraform, Azure CLI, Git, VS Code, and the recommended extensions form the Windows-first control surface. When those tools are present and consistent, the operator has a reliable place to run checks, read plans, inspect outputs, and clean up resources. The local machine becomes part of the cloud operating system.

The foundation work gives Terraform its vocabulary. Resource groups define boundaries. Tags identify project, environment, ownership, and lab context. Variables separate inputs from implementation. Locals keep names and tag composition consistent. Outputs expose the values needed for validation. State records what Terraform manages. Locking protects that record when more than one operation might touch it.

The infrastructure layers turn that vocabulary into an environment. Virtual networks provide address space. Subnets divide roles. Network Security Groups express traffic intent. Windows Server and IIS create a visible service. Bastion keeps administrative access away from default public RDP. Load balancers, NAT rules, VM scale sets, and autoscale settings show how a Windows workload can move from a single server to a managed tier.

The platform and operations patterns add stronger control. NAT Gateway gives private workloads predictable outbound access. Private DNS gives internal names a managed resolution path. Public DNS stays optional because domain ownership is not a lab assumption. Remote state supports shared Terraform work. Global routing introduces resilient HTTP entry points. Key Vault keeps generated secrets out of plain text. Private endpoints reduce public exposure for managed services. Log Analytics gives operations a workspace for evidence. GitHub Actions creates a repeatable Terraform review path. Azure SQL private access shows how database access can depend on private networking and private DNS instead of a public endpoint.

The final reference architecture matters because it composes those parts. A Windows application tier needs more than compute. It needs network placement, a load balancer, generated credentials, administrative access, secret storage, monitoring, outputs, validation, and

teardown. Terraform turns those relationships into a graph that can be read before it changes Azure.

The core loop stays deliberately small:

```
terraform init
terraform fmt -check
terraform validate
terraform plan -out tfplan
terraform apply tfplan
terraform output
terraform destroy
```

`terraform init` prepares the folder and providers. `terraform fmt -check` keeps the codebase consistent. `terraform validate` checks structure and provider schema. `terraform plan -out tfplan` creates a review point. `terraform apply tfplan` applies the exact reviewed plan. `terraform output` supports validation. `terraform destroy` removes what should not keep running.

The repository also includes checks for the wider working surface:

```
.\scripts\Test-AzureFromZeroToHeroToolchain.ps1
.\scripts\Test-AzureFromZeroToHeroTerraform.ps1
.\scripts\Test-AzureFromZeroToHeroTerraform.ps1 -Validate
```

The toolchain script checks the local commands and recommended VS Code extensions. The Terraform script checks recursive formatting. With validation enabled, it initializes lab folders with backend disabled and runs `terraform validate`. These checks do not replace plan review, Azure authentication, quota awareness, or design judgment. They make the basic failure modes visible earlier.

Security discipline is built from repeated small choices. Real `terraform.tfvars` files stay local. Safe examples live in `terraform.tfvars.example`. Generated passwords are treated as secrets. Sensitive outputs stay sensitive. Key Vault secret names can be documented, but secret values should not be copied into manuscript files, tickets, screenshots, or commits. Tenant IDs, object IDs, subscription IDs, client secrets, SQL credentials, connection strings, workflow credentials, and workspace keys do not belong in learning artifacts.

Cleanup is part of the design. Labs can leave behind cost and clutter: VMs, public IPs, load balancers, databases, DNS zones, private endpoints, workspaces, storage accounts, and monitoring resources. A lab is not complete just because the resource exists. It is complete when the validation path is clear and the teardown path is safe.

Source control gives cloud work a place to be questioned before it becomes real infrastructure. A Terraform change can be reviewed as a diff. A pull request can run format, validation, and plan checks. A failed workflow can point to the layer that needs attention: trigger path, working directory, provider initialization, formatting, validation, plan input, or Azure authentication.

This is the practical value of Infrastructure as Code. It moves cloud changes from private memory into shared review.

The architecture can grow from here, but it should grow through the same discipline. Separate environments, stricter remote-state design, module versioning, policy checks, diagnostic settings, alert receivers, managed identities, backup policies, private DNS forwarding, stronger CI gates, and deployment approvals are all reasonable extension paths. They are not automatic. They should be introduced as explicit, validated changes with clear rollback and cleanup behavior.

The lasting habit is simple: write the desired state, format it, validate it, read the plan, apply only what was reviewed, inspect the result, and remove what should not remain. Azure resources are the visible output. The deeper result is the ability to operate Azure as code.